HSK 语音学习系统

版本号：V1.0

源代码

目录

■■ 2
■■ 3
server/server.js 3
server/app.js 5
server/database.js 10
server/ai-client.js 15
server/asr-client.js 16
server/auth.js 17
server/password.js 18
apps/shared/app.js 18
apps/shared/audio-recorder.js 24
apps/shared/speech-engine.js 25
apps/platform/platform.js 26
android_apps/hsk_dialogue_studio/lib/app.dart 29
android_apps/hsk_dialogue_studio/lib/coach_api.dart 43
android_apps/hsk_mission_lab/lib/app.dart 48
android_apps/hsk_mission_lab/lib/coach_api.dart 62
database/migrations/003_practice_platform.sql 66

版本号：V1.0

> 状态：草稿/需人工确认。源码摘录按当前工作区生成，提交前需由申请人确认开发者、首次发表日期和最终版本范围。

目录

- ☐ server/server.js
- ☐ server/app.js
- ☐ server/database.js
- ☐ server/ai-client.js
- ☐ server/asr-client.js
- ☐ server/auth.js
- ☐ server/password.js
- ☐ apps/shared/app.js
- ☐ apps/shared/audio-recorder.js
- ☐ apps/shared/speech-engine.js
- ☐ apps/platform/platform.js
- ☐ android_apps/hsk_dialogue_studio/lib/app.dart
- ☐ android_apps/hsk_dialogue_studio/lib/coach_api.dart
- ☐ android_apps/hsk_mission_lab/lib/app.dart
- ☐ android_apps/hsk_mission_lab/lib/coach_api.dart
- ☐ database/migrations/001_init.sql
- ☐ database/migrations/002_oidc.sql
- ☐ database/migrations/003_practice_platform.sql

server/server.js

```
import { randomBytes } from 'node:crypto';
import { readFile } from 'node:fs/promises';

import { createBailianClient } from './ai-client.js';
import { createBailianAsrClient } from './asr-client.js';
import { createAppServer } from './app.js';
import { createDatabasePool, createPostgresStore } from './database.js';
import { createWebServer } from './web-server.js';
```

```
import { createOidcVerifier } from './oidc.js';

const host = process.env.HOST || '***';
const port = Number(process.env.PORT || 8040);
const webPort = Number(process.env.WEB_PORT || 4035);
const tlsKeyPath = process.env.TLS_KEY_PATH || '';
const tlsCertPath = process.env.TLS_CERT_PATH || '';
if (Boolean(tlsKeyPath) !== Boolean(tlsCertPath)) {
  throw new Error('TLS_KEY_PATH and TLS_CERT_PATH must be configured together');
}
const tlsOptions = tlsKeyPath
  ? { key: await readFile(tlsKeyPath), cert: await readFile(tlsCertPath) }
  : null;
const aiApiKey = process.env.AI_API_KEY || process.env.DASHSCOPE_API_KEY || '';
const aiApiUrl = process.env.AI_BASE_URL || process.env.DASHSCOPE_API_URL;
const aiModel = process.env.AI_MODEL || process.env.DASHSCOPE_MODEL || 'qwen-plus';
const aiProviderName = process.env.AI_BASE_URL ? '    AI' : '  ';
const tokenSecret = process.env.TOKEN_SECRET || randomBytes(32).toString('hex');
const aiClient = createBailianClient({
  apiKey: aiApiKey,
  apiUrl: aiApiUrl,
  model: aiModel,
  providerName: aiProviderName,
});
const asrClient = createBailianAsrClient({
  apiKey: process.env.DASHSCOPE_API_KEY || '',
  apiUrl: process.env.DASHSCOPE_API_URL,
  model: process.env.DASHSCOPE_ASR_MODEL || 'qwen3-asr-flash',
});
const databasePool = createDatabasePool();
const store = databasePool ? createPostgresStore(databasePool) : null;
await store?.ping();
const oidcIssuer = (process.env.OIDC_ISSUER || '').replace(/\/$/, '');
const oidcVerifier = createOidcVerifier({
  issuer: oidcIssuer,
  clientIds: (process.env.OIDC_CLIENT_IDS || '').split(',').map((value) => value.trim()).filter(Boolean),
});
const server = createAppServer({
  tokenSecret,
  aiClient,
  asrClient,
  store,
  aiConfigured: Boolean(aiApiKey),
  oidcVerifier,
  oidcIssuer,
  tlsOptions,
});

const webServer = createWebServer(port, {
  tlsOptions,
  apiTls: Boolean(tlsOptions),
  apiHost: process.env.WEB_API_HOST || '***',
});
for (const listener of [server, webServer]) {
  listener.on('error', (error) => {
    console.error(`HSK        ${error.code}`);
    process.exit(1);
  });
}
server.listen(port, host, () => {
  console.log(`HSK API: ${tlsOptions ? 'https' : 'http'}://${host}:${port}`);
  console.log(`AI configured: ${aiApiKey ? 'yes' : 'no'}`);
  console.log(`Database configured: ${store ? 'yes' : 'no'}`);
});
webServer.listen(webPort, host, () => {
  console.log(`HSK Voice Learning: ${tlsOptions ? 'https' : 'http'}://${host}:${webPort}`);
});

function shutdown() {
  webServer.close();
  server.close(async () => {
    await store?.close();
    process.exit(0);
  });
}

process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);
```

server/app.js

```
import { createReadStream } from 'node:fs';
import { stat } from 'node:fs/promises';
import { createServer } from 'node:http';
import { createServer as createSecureServer } from 'node:https';
import { extname, join, normalize, resolve } from 'node:path';
import { fileURLToPath } from 'node:url';

import { issueToken, verifyToken } from './auth.js';
import { ALL_DEMO_ACCOUNTS } from './demo-accounts.js';
import { BUILT_IN_QUESTIONS, PRACTICE_SOURCES } from './practice-content.js';

const PROJECT_ROOT = resolve(fileURLToPath(new URL('..', import.meta.url)));
const STATIC_ROUTES = [
  ['/platform/', join(PROJECT_ROOT, 'apps', 'platform')],
  ['/studio/', join(PROJECT_ROOT, 'apps', 'dialogue-studio')],
  ['/mission/', join(PROJECT_ROOT, 'apps', 'mission-lab')],
  ['/shared/', join(PROJECT_ROOT, 'apps', 'shared')],
];
const MIME_TYPES = {
  '.css': 'text/css; charset=utf-8',
  '.html': 'text/html; charset=utf-8',
  '.ico': 'image/x-icon',
  '.js': 'text/javascript; charset=utf-8',
  '.json': 'application/json; charset=utf-8',
  '.png': 'image/png',
  '.svg': 'image/svg+xml',
  '.webmanifest': 'application/manifest+json',
};

// Local demo identities keep the APK and browser smoke flows usable before a
// PostgreSQL instance is provisioned. They are intentionally only active when
// no database store is configured.
const DEMO_IDENTITIES = new Map(ALL_DEMO_ACCOUNTS.map((account) => [account.username, account]));

function sendJson(response, status, body) {
  const payload = JSON.stringify(body);
  response.writeHead(status, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(payload),
    'Cache-Control': 'no-store',
  });
  response.end(payload);
}

async function readJson(request, limit = 256 * 1024) {
  const chunks = [];
  let size = 0;
  for await (const chunk of request) {
    size += chunk.length;
    if (size > limit) throw new Error('REQUEST_TOO_LARGE');
    chunks.push(chunk);
  }
  if (!chunks.length) return {};
  return JSON.parse(Buffer.concat(chunks).toString('utf8'));
}

function getBearerToken(request) {
  const match = /^Bearer\s+(.+)$/i.exec(request.headers.authorization || '');
  return match?.[1] || '';
}

async function serveStatic(pathname, response) {
  const route = STATIC_ROUTES.find(([prefix]) => pathname.startsWith(prefix));
  if (!route) return false;
  const [prefix, root] = route;
  const relativePath = pathname.slice(prefix.length) || 'index.html';
  const cleanPath = normalize(relativePath).replace(/^(\.\.(?:[\\/]|$))+/, '');
  const filePath = resolve(root, cleanPath);
  if (!filePath.startsWith(resolve(root))) return false;

  try {
    const fileStat = await stat(filePath);
```

```
    const finalPath = fileStat.isDirectory() ? join(filePath, 'index.html') : filePath;
    const finalStat = fileStat.isDirectory() ? await stat(finalPath) : fileStat;
    response.writeHead(200, {
      'Content-Type': MIME_TYPES[extname(finalPath).toLowerCase()] || 'application/octet-stream',
      'Content-Length': finalStat.size,
      'Cache-Control': extname(finalPath) === '.html' ? 'no-cache' : 'public, max-age=3600',
    });
    createReadStream(finalPath).pipe(response);
    return true;
  } catch {
    return false;
  }
}

export function createAppServer({
  tokenSecret,
  aiClient,
  asrClient = null,
  store = null,
  aiConfigured = false,
  oidcVerifier = null,
  oidcIssuer = '',
  tlsOptions = null,
  allowDemoLogin = process.env.NODE_ENV !== 'production',
} = {}) {
  if (!tokenSecret) throw new Error('createAppServer requires tokenSecret');
  if (!aiClient?.chat) throw new Error('createAppServer requires aiClient.chat');

  const memoryQuestions = [];
  const memoryAttempts = new Map();
  const handler = async (request, response) => {
    response.setHeader('X-Content-Type-Options', 'nosniff');
    response.setHeader('Referrer-Policy', 'same-origin');
    response.setHeader('Permissions-Policy', 'microphone=(self)');
    response.setHeader('Content-Security-Policy', "default-src 'self'; connect-src 'self'; script-src 'self';
style-src 'self'; img-src 'self' data:; font-src 'self'; media-src 'self' blob:");

    const url = new URL(request.url || '/', 'http://localhost');
    const { pathname } = url;

    try {
      if (request.method === 'GET' && pathname === '/') {
        response.writeHead(302, { Location: '/studio/' });
        response.end();
        return;
      }

      if (request.method === 'GET' && pathname === '/api/health') {
        sendJson(response, 200, { ok: true, aiConfigured, service: 'hsk-voice-learning' });
        return;
      }

      if (pathname.startsWith('/api/practice/') || pathname === '/api/admin/questions') {
        const session = verifyToken(getBearerToken(request), tokenSecret);
        if (!session) {
          sendJson(response, 401, { error: '           ' });
          return;
        }

        if (request.method === 'GET' && pathname === '/api/practice/questions') {
          const level = Number(url.searchParams.get('level'));
          const section = url.searchParams.get('section');
          if (!Number.isInteger(level) || level < 1 || level > 6 || !['listening', 'reading', 'speaking'].includes(
section)) {
            sendJson(response, 400, { error: '         ' });
            return;
          }
          const published = store?.listPracticeQuestions
            ? await store.listPracticeQuestions({ level, section })
            : memoryQuestions.filter((item) => item.level === level && item.section === section);
          sendJson(response, 200, {
            questions: [...BUILT_IN_QUESTIONS.filter((item) => item.level === level && item.section === section), .
..published]
              .map(({ answer: _answer, ...question }) => question),
            sources: PRACTICE_SOURCES,
          });
          return;
        }
```

```
        if (request.method === 'POST' && pathname === '/api/admin/questions') {
          if (session.role !== 'teacher') {
            sendJson(response, 403, { error: '          ' });
            return;
          }
          const body = await readJson(request);
          const level = Number(body.level);
          const options = Array.isArray(body.options) ? body.options.map((option) => String(option).trim()) : [];
          const answer = Number(body.answer);
          const question = String(body.question || '').trim();
          const section = String(body.section || '');
          if (!Number.isInteger(level) || level < 1 || level > 6
            || !['listening', 'reading', 'speaking'].includes(section)
            || question.length < 4 || question.length > 4000
            || options.length < 2 || options.length > 6
            || options.some((option) => !option || option.length > 500)
            || !Number.isInteger(answer) || answer < 1 || answer > options.length) {
            sendJson(response, 400, { error: '              ' });
            return;
          }
          const input = {
            level, section, question, options, answer,
            explanation: String(body.explanation || '').slice(0, 2000),
            audioText: String(body.audioText || '').slice(0, 4000),
            source: String(body.source || '     ').slice(0, 300),
            createdBy: session.sub,
          };
          const published = store?.createPracticeQuestion
            ? await store.createPracticeQuestion(input)
            : { ...input, id: `admin-${Date.now()}-${Math.random().toString(36).slice(2, 8)}` };
          if (!store?.createPracticeQuestion) memoryQuestions.push(published);
          sendJson(response, 201, { question: published });
          return;
        }

        if (request.method === 'POST' && pathname === '/api/practice/attempts') {
          const body = await readJson(request);
          const level = Number(body.level);
          const answer = Number(body.answer);
          const section = String(body.section || '');
          const published = store?.listPracticeQuestions
            ? await store.listPracticeQuestions({ level, section })
            : memoryQuestions.filter((item) => item.level === level && item.section === section);
          const available = [...BUILT_IN_QUESTIONS, ...published];
          const question = available.find((item) => item.id === body.questionId && item.level === level && item.
section === section);
          if (!question || !Number.isInteger(answer) || answer < 1 || answer > question.options.length) {
            sendJson(response, 400, { error: '       ' });
            return;
          }
          const correct = answer === question.answer;
          const attempt = store?.savePracticeAttempt
            ? await store.savePracticeAttempt({ userId: session.sub, questionId: question.id, level, section,
correct })
            : { correct };
          if (!store?.savePracticeAttempt) {
            const attempts = memoryAttempts.get(session.sub) || [];
            attempts.push({ section, correct });
            memoryAttempts.set(session.sub, attempts);
          }
          sendJson(response, 200, {
            correct,
            correctAnswer: correct ? null : question.answer,
            explanation: question.explanation,
            attempt,
          });
          return;
        }

        if (request.method === 'GET' && pathname === '/api/practice/stats') {
          const attempts = memoryAttempts.get(session.sub) || [];
          const stats = store?.getPracticeStats
            ? await store.getPracticeStats(session.sub)
            : {
                total: attempts.length,
                correct: attempts.filter((item) => item.correct).length,
                bySection: Object.fromEntries(['listening', 'reading', 'speaking'].map((section) => {
                  const sectionAttempts = attempts.filter((item) => item.section === section);
```

```
                  return [section, {
                    total: sectionAttempts.length,
                    correct: sectionAttempts.filter((item) => item.correct).length,
                    rate: sectionAttempts.length
                      ? Math.round(sectionAttempts.filter((item) => item.correct).length / sectionAttempts.length
* 100)
                      : 0,
                  }];
                })),
              };
          sendJson(response, 200, { ...stats, sources: PRACTICE_SOURCES });
          return;
        }
      }

      if (request.method === 'POST' && pathname === '/api/login') {
        const body = await readJson(request);
        const normalizedUsername = String(body.username || '').trim().toLowerCase();
        const demo = allowDemoLogin ? DEMO_IDENTITIES.get(normalizedUsername) : null;
        const authenticated = await store?.authenticate?.(body.username, body.password);
        const user = authenticated || allowDemoLogin
          ? authenticated || (demo && String(body.password || '') === demo.password
            ? (({ password: _password, ...identity }) => identity)(demo)
            : null)
          : null;
        await store?.recordLogin?.({
          userId: authenticated?.id,
          username: body.username,
          success: Boolean(user),
          clientApp: body.appId,
          remoteAddress: request.socket.remoteAddress,
        });
        if (!user) {
          sendJson(response, 401, { error: '       ' });
          return;
        }
        sendJson(response, 200, { user, token: issueToken(user, tokenSecret) });
        return;
      }

      if (request.method === 'POST' && pathname === '/api/auth/oidc') {
        if (!oidcVerifier || !store?.authenticateOidc) {
          sendJson(response, 503, { error: '        ' });
          return;
        }
        const body = await readJson(request);
        let identity;
        try {
          identity = await oidcVerifier(String(body.accessToken || ''));
        } catch {
          sendJson(response, 401, { error: '            ' });
          return;
        }
        const user = await store.authenticateOidc({ ...identity, issuer: oidcIssuer });
        if (user?.conflict) {
          sendJson(response, 409, { error: '                  ' });
          return;
        }
        if (!user) {
          sendJson(response, 403, { error: '            ' });
          return;
        }
        await store.recordLogin?.({
          userId: user.id,
          username: user.username,
          success: true,
          clientApp: body.appId,
          remoteAddress: request.socket.remoteAddress,
        });
        sendJson(response, 200, { user, token: issueToken(user, tokenSecret) });
        return;
      }

      if (request.method === 'POST' && pathname === '/api/chat') {
        const session = verifyToken(getBearerToken(request), tokenSecret);
        if (!session) {
          sendJson(response, 401, { error: '           ' });
          return;
```

```
    }
    const body = await readJson(request);
    if (!Array.isArray(body.messages) || !body.messages.length) {
      sendJson(response, 400, { error: '       ' });
      return;
    }
    const reply = await aiClient.chat({
      level: body.level,
      scenario: body.scenario,
      messages: body.messages,
    });
    const userMessage = [...body.messages]
      .reverse()
      .find((message) => message?.role === 'user' && String(message.content || '').trim());
    const persisted = userMessage && !String(session.sub).startsWith('demo-') && store?.saveChatTurn
      ? await store.saveChatTurn({
        conversationId: body.conversationId,
        userId: session.sub,
        clientApp: body.appId,
        level: body.level,
        scenario: body.scenario,
        source: userMessage.source,
        userMessage: String(userMessage.content).trim(),
        assistantMessage: reply,
      })
      : null;
    sendJson(response, 200, {
      reply,
      conversationId: persisted?.conversationId || body.conversationId || null,
    });
    return;
  }

  if (request.method === 'POST' && pathname === '/api/transcribe') {
    const session = verifyToken(getBearerToken(request), tokenSecret);
    if (!session) {
      sendJson(response, 401, { error: '           ' });
      return;
    }
    if (!asrClient?.transcribe) {
      sendJson(response, 503, { error: '          ' });
      return;
    }
    const body = await readJson(request, 14 * 1024 * 1024);
    const audio = String(body.audio ?? '');
    if (!/^data:audio\/wav;base64,[A-Za-z0-9+/=]+$/i.test(audio)) {
      sendJson(response, 400, { error: '    WAV     ' });
      return;
    }
    const base64Length = audio.length - audio.indexOf(',') - 1;
    if (base64Length * 0.75 > 10 * 1024 * 1024) {
      sendJson(response, 413, { error: '         10 MB' });
      return;
    }
    const transcript = await asrClient.transcribe({
      audioDataUri: audio,
      locale: body.locale,
    });
    const persisted = !String(session.sub).startsWith('demo-') && store?.saveTranscription
      ? await store.saveTranscription({
        conversationId: body.conversationId,
        userId: session.sub,
        clientApp: body.appId,
        level: body.level,
        scenario: body.scenario,
        transcript,
        locale: body.locale,
        audioBytes: Math.max(1, Math.floor(base64Length * 0.75)),
      })
      : null;
    sendJson(response, 200, {
      transcript,
      conversationId: persisted?.conversationId || body.conversationId || null,
    });
    return;
  }

  if (request.method === 'GET' && pathname === '/api/history/latest') {
```

```
        const session = verifyToken(getBearerToken(request), tokenSecret);
        if (!session) {
          sendJson(response, 401, { error: '           ' });
          return;
        }
        const rawLimit = url.searchParams.get('limit');
        const limit = rawLimit == null ? 80 : Number(rawLimit);
        if (!Number.isInteger(limit) || limit < 1 || limit > 100) {
          sendJson(response, 400, { error: '         ' });
          return;
        }
        const cursorParts = [
          url.searchParams.get('cursorConversationId'),
          url.searchParams.get('cursorId'),
          url.searchParams.get('cursorCreatedAt'),
        ];
        const hasCursorPart = cursorParts.some((value) => value != null);
        const [conversationId, id, createdAt] = cursorParts;
        const validCursor = !hasCursorPart || (
          /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i.test(conversationId || '')
          && /^[1-9]\d{0,18}$/.test(id || '')
          && BigInt(id) <= 9223372036854775807n
          && Number.isFinite(Date.parse(createdAt || ''))
        );
        if (!validCursor) {
          sendJson(response, 400, { error: '         ' });
          return;
        }
        const history = await store?.getLatestConversation?.({
          userId: session.sub,
          clientApp: url.searchParams.get('appId'),
          limit,
          cursor: hasCursorPart ? { conversationId, id, createdAt } : null,
        });
        sendJson(response, 200, history || {
          conversationId: null,
          messages: [],
          hasMore: false,
          nextCursor: null,
        });
        return;
      }

      if (request.method === 'GET' && await serveStatic(pathname, response)) return;
      sendJson(response, 404, { error: '        ' });
    } catch (error) {
      if (error?.message === 'REQUEST_TOO_LARGE') {
        sendJson(response, 413, { error: '      ' });
        return;
      }
      if (error instanceof SyntaxError) {
        sendJson(response, 400, { error: '       ' });
        return;
      }
      const safeMessage = error?.name === 'TimeoutError'
        ? 'AI           '
        : String(error?.message || '        ').replace(/sk-[a-zA-Z0-9]+/g, '[   ]');
      sendJson(response, 502, { error: safeMessage });
    }
  };
  return tlsOptions ? createSecureServer(tlsOptions, handler) : createServer(handler);
}
```

server/database.js

```
import { randomBytes, randomUUID } from 'node:crypto';

import pg from 'pg';

import { verifyPassword } from './password.js';

const { Pool } = pg;

function clampLevel(value) {
  const level = Number(value);
```

```
  return Number.isInteger(level) && level >= 1 && level <= 6 ? level : 3;
}

function normalizeApp(value) {
  return String(value || 'unknown').trim().slice(0, 120) || 'unknown';
}

function normalizeScenario(value) {
  return String(value || '    ').trim().slice(0, 120) || '    ';
}

function validUuid(value) {
  return /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i.test(String(value || ''));
}

export function createDatabasePool(env = process.env) {
  if (!env.DB_HOST || !env.DB_USER || !env.DB_PASSWORD || !env.DB_NAME) return null;
  return new Pool({
    host: env.DB_HOST,
    port: Number(env.DB_PORT || 5432),
    user: env.DB_USER,
    password: env.DB_PASSWORD,
    database: env.DB_NAME,
    max: Number(env.DB_POOL_SIZE || 10),
    idleTimeoutMillis: 30_000,
    connectionTimeoutMillis: 5_000,
    ssl: env.DB_SSL === 'true' ? { rejectUnauthorized: false } : false,
  });
}

async function ensureConversation(client, input) {
  if (validUuid(input.conversationId)) {
    const existing = await client.query(
      'SELECT id FROM learning_conversations WHERE id = $1 AND user_id = $2',
      [input.conversationId, input.userId],
    );
    if (existing.rowCount) return existing.rows[0].id;
  }

  const conversationId = randomUUID();
  await client.query(
    `INSERT INTO learning_conversations
      (id, user_id, client_app, hsk_level, scenario)
     VALUES ($1, $2, $3, $4, $5)`,
    [
      conversationId,
      input.userId,
      normalizeApp(input.clientApp),
      clampLevel(input.level),
      normalizeScenario(input.scenario),
    ],
  );
  return conversationId;
}

export function createPostgresStore(pool) {
  if (!pool?.query || !pool?.connect) throw new Error('PostgreSQL pool is required');

  return {
    async ping() {
      await pool.query('SELECT 1');
    },

    async authenticate(username, password) {
      const normalized = String(username || '').trim().toLowerCase();
      const result = await pool.query(
        `SELECT id, username, display_name, role, password_salt, password_hash
         FROM app_users WHERE username = $1 AND is_active = true`,
        [normalized],
      );
      if (!result.rowCount) return null;
      const row = result.rows[0];
      const valid = await verifyPassword(password, {
        salt: row.password_salt,
        hash: row.password_hash,
      });
      if (!valid) return null;
      return {
```

```
        id: row.id,
        username: row.username,
        displayName: row.display_name,
        role: row.role,
      };
    },

    async authenticateOidc({ subject, issuer, username, displayName }) {
      const client = await pool.connect();
      try {
        await client.query('BEGIN');
        await client.query('SELECT pg_advisory_xact_lock(hashtext($1))', [`hsk-oidc:${subject}`]);
        const linked = await client.query(
          `SELECT u.id, u.username, u.display_name, u.role
           FROM hsk_oidc_identities i JOIN app_users u ON u.id = i.user_id
           WHERE i.subject = $1 AND i.issuer = $2 AND u.is_active = true`,
          [subject, issuer],
        );
        let user;
        if (linked.rowCount) {
          user = linked.rows[0];
          await client.query('UPDATE app_users SET display_name = $2, updated_at = now() WHERE id = $1', [user.id,
displayName]);
        } else {
          const collision = await client.query('SELECT 1 FROM app_users WHERE username = $1', [username]);
          if (collision.rowCount) {
            await client.query('ROLLBACK');
            return { conflict: true };
          }
          const id = randomUUID();
          const opaquePassword = randomBytes(48).toString('base64');
          const created = await client.query(
            `INSERT INTO app_users
              (id, username, display_name, role, password_salt, password_hash)
             VALUES ($1, $2, $3, 'student', $4, $5)
             RETURNING id, username, display_name, role`,
            [id, username, displayName, randomBytes(16).toString('base64'), opaquePassword],
          );
          user = created.rows[0];
          await client.query(
            'INSERT INTO hsk_oidc_identities (subject, user_id, issuer) VALUES ($1, $2, $3)',
            [subject, id, issuer],
          );
        }
        await client.query('COMMIT');
        return {
          id: user.id,
          username: user.username,
          displayName: user.display_name,
          role: user.role,
        };
      } catch (error) {
        await client.query('ROLLBACK').catch(() => {});
        throw error;
      } finally {
        client.release();
      }
    },

    async recordLogin({ userId, username, success, clientApp, remoteAddress }) {
      const address = String(remoteAddress || '').replace(/^::ffff:/, '');
      await pool.query(
        `INSERT INTO login_events
          (user_id, attempted_username, success, client_app, remote_address)
         VALUES ($1, $2, $3, $4, NULLIF($5, '')::inet)`,
        [userId || null, String(username || '').slice(0, 120), success, normalizeApp(clientApp), address],
      );
    },

    async saveTranscription(input) {
      const client = await pool.connect();
      try {
        await client.query('BEGIN');
        const conversationId = await ensureConversation(client, input);
        await client.query(
          `INSERT INTO transcription_events
            (id, conversation_id, user_id, transcript, locale, audio_bytes)
           VALUES ($1, $2, $3, $4, $5, $6)`,
```

```
          [
            randomUUID(),
            conversationId,
            input.userId,
            input.transcript,
            String(input.locale || 'zh-CN').slice(0, 20),
            input.audioBytes,
          ],
        );
        await client.query(
          'UPDATE learning_conversations SET last_active_at = now() WHERE id = $1',
          [conversationId],
        );
        await client.query('COMMIT');
        return { conversationId };
      } catch (error) {
        await client.query('ROLLBACK');
        throw error;
      } finally {
        client.release();
      }
    },

    async saveChatTurn(input) {
      const client = await pool.connect();
      try {
        await client.query('BEGIN');
        const conversationId = await ensureConversation(client, input);
        await client.query(
          `INSERT INTO conversation_messages
            (conversation_id, user_id, role, source, content)
           VALUES ($1, $2, 'user', $3, $4),
                  ($1, $2, 'assistant', 'ai', $5)`,
          [conversationId, input.userId, input.source === 'voice' ? 'voice' : 'text', input.userMessage, input.
assistantMessage],
        );
        await client.query(
          `UPDATE learning_conversations
           SET hsk_level = $2, scenario = $3, last_active_at = now()
           WHERE id = $1`,
          [conversationId, clampLevel(input.level), normalizeScenario(input.scenario)],
        );
        await client.query('COMMIT');
        return { conversationId };
      } catch (error) {
        await client.query('ROLLBACK');
        throw error;
      } finally {
        client.release();
      }
    },

    async getLatestConversation({ userId, clientApp, limit = 80, cursor = null }) {
      const pageSize = Number.isInteger(Number(limit))
        ? Math.min(100, Math.max(1, Number(limit)))
        : 80;
      let conversation;
      if (cursor) {
        if (
          !validUuid(cursor.conversationId)
          || !/^[1-9]\d{0,18}$/.test(String(cursor.id))
          || BigInt(cursor.id) > 9223372036854775807n
          || !Number.isFinite(Date.parse(cursor.createdAt))
        ) {
          return null;
        }
        conversation = await pool.query(
          `SELECT id, hsk_level, scenario
           FROM learning_conversations
           WHERE id = $1 AND user_id = $2 AND client_app = $3
           LIMIT 1`,
          [cursor.conversationId, userId, normalizeApp(clientApp)],
        );
      } else {
        conversation = await pool.query(
          `SELECT id, hsk_level, scenario
           FROM learning_conversations
           WHERE user_id = $1 AND client_app = $2
```

```
        ORDER BY last_active_at DESC, id DESC LIMIT 1`,
       [userId, normalizeApp(clientApp)],
     );
   }
   if (!conversation.rowCount) return null;
   const row = conversation.rows[0];
   const messages = await pool.query(
     `SELECT id::text AS id, role, content, source, created_at,
             created_at::text AS cursor_created_at
      FROM conversation_messages
      WHERE conversation_id = $1
        ${cursor ? 'AND (created_at, id) < ($2::timestamptz, $3::bigint)' : ''}
      ORDER BY created_at DESC, id DESC
      LIMIT $${cursor ? 4 : 2}`,
     cursor
       ? [row.id, cursor.createdAt, cursor.id, pageSize + 1]
       : [row.id, pageSize + 1],
   );
   const hasMore = messages.rows.length > pageSize;
   const pageRows = messages.rows.slice(0, pageSize);
   const oldest = pageRows.at(-1);
   return {
     conversationId: row.id,
     level: row.hsk_level,
     scenario: row.scenario,
     hasMore,
     nextCursor: hasMore && oldest
       ? {
           conversationId: row.id,
           id: oldest.id,
           createdAt: oldest.cursor_created_at,
         }
       : null,
     messages: pageRows.reverse().map((message) => ({
       role: message.role,
       content: message.content,
       source: message.source,
       createdAt: message.created_at,
     })),
   };
 },

 async listPracticeQuestions({ level, section }) {
   const result = await pool.query(
     `SELECT id, level, section, question, options, answer, explanation,
             audio_text AS "audioText", source
      FROM hsk_practice_questions
      WHERE level = $1 AND section = $2
      ORDER BY created_at DESC, id`,
     [level, section],
   );
   return result.rows;
 },

 async createPracticeQuestion(input) {
   const id = randomUUID();
   const result = await pool.query(
     `INSERT INTO hsk_practice_questions
       (id, level, section, question, options, answer, explanation, audio_text, source, created_by)
      VALUES ($1, $2, $3, $4, $5::jsonb, $6, $7, $8, $9, $10)
      RETURNING id, level, section, question, options, answer, explanation,
                audio_text AS "audioText", source`,
     [id, input.level, input.section, input.question, JSON.stringify(input.options), input.answer,
       input.explanation, input.audioText, input.source, input.createdBy],
   );
   return result.rows[0];
 },

 async savePracticeAttempt(input) {
   const result = await pool.query(
     `INSERT INTO hsk_practice_attempts (user_id, question_id, level, section, is_correct)
      VALUES ($1, $2, $3, $4, $5)
      RETURNING id, is_correct AS correct, attempted_at`,
     [input.userId, input.questionId, input.level, input.section, input.correct],
   );
   return result.rows[0];
 },
 async getPracticeStats(userId) {
```

```
      const result = await pool.query(
        `SELECT count(*)::int AS total,
                count(*) FILTER (WHERE is_correct)::int AS correct,
                section,
                count(*)::int AS section_total,
                count(*) FILTER (WHERE is_correct)::int AS section_correct,
                count(*) FILTER (WHERE is_correct)::int * 100.0 / NULLIF(count(*), 0) AS section_rate
           FROM hsk_practice_attempts WHERE user_id = $1
           GROUP BY GROUPING SETS ((), (section))`,
        [userId],
      );
      const summary = result.rows.find((row) => row.section == null) || { total: 0, correct: 0 };
      return {
        total: summary.total,
        correct: summary.correct,
        bySection: Object.fromEntries(result.rows.filter((row) => row.section).map((row) => [
          row.section, { total: row.section_total, rate: Number(row.section_rate || 0) },
        ])),
      };
    },

    async close() {
      await pool.end();
    },
  };
}
```

server/ai-client.js

```
const DEFAULT_API_URL = 'https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions';
const MAX_HISTORY_MESSAGES = 24;

function normalizeLevel(level) {
  const numericLevel = Number(level);
  return Number.isInteger(numericLevel) && numericLevel >= 1 && numericLevel <= 6
    ? numericLevel
    : 3;
}

function normalizeScenario(scenario) {
  const value = String(scenario ?? '').trim();
  return value.slice(0, 80) || '    ';
}

export function buildMessages({ level, scenario, messages }) {
  const safeLevel = normalizeLevel(level);
  const safeScenario = normalizeScenario(scenario);
  const validHistory = Array.isArray(messages)
    ? messages
      .filter((message) => ['user', 'assistant'].includes(message?.role))
      .map((message) => ({
        role: message.role,
        content: String(message.content ?? '').trim().slice(0, 2000),
      }))
      .filter((message) => message.content)
      .slice(-MAX_HISTORY_MESSAGES)
    : [];

  return [
    {
      role: 'system',
      content: [
        `        HSK ${safeLevel}        `,
        `       “${safeScenario}” `,
        '                          2   4   ',
        '                                  ',
        '              “                “ “                    ” ',
        '     Markdown                           ',
      ].join(''),
    },
    ...validHistory,
  ];
}

export function createBailianClient({
```

```
  apiKey,
  apiUrl = DEFAULT_API_URL,
  model = 'qwen-plus',
  providerName = '  ',
  fetchImpl = globalThis.fetch,
  timeoutMs = 45_000,
} = {}) {
  return {
    async chat(input) {
      if (!apiKey) throw new Error('        AI_API_KEY   DASHSCOPE_API_KEY');
      if (typeof fetchImpl !== 'function') throw new Error('   Node        fetch');

      const response = await fetchImpl(apiUrl, {
        method: 'POST',
        headers: {
          Authorization: `Bearer ${apiKey}`,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          model,
          messages: buildMessages(input),
          temperature: 0.65,
          max_tokens: 320,
        }),
        signal: AbortSignal.timeout(timeoutMs),
      });

      if (!response.ok) {
        let detail = '';
        try {
          const payload = await response.json();
          detail = payload?.error?.message || payload?.message || '';
        } catch {
          detail = '';
        }
        throw new Error(detail || `${providerName}     HTTP ${response.status} `);
      }

      const payload = await response.json();
      const reply = payload?.choices?.[0]?.message?.content;
      if (typeof reply !== 'string' || !reply.trim()) {
        throw new Error(`${providerName}      `);
      }
      return reply.trim();
    },
  };
}

export const bailianDefaults = {
  apiUrl: DEFAULT_API_URL,
  maxHistoryMessages: MAX_HISTORY_MESSAGES,
};
```

server/asr-client.js

```
const DEFAULT_API_URL = 'https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions';

function languageFromLocale(locale) {
  const normalized = String(locale ?? '').toLowerCase();
  if (normalized.startsWith('zh')) return 'zh';
  if (normalized.startsWith('yue')) return 'yue';
  return undefined;
}

export function createBailianAsrClient({
  apiKey,
  apiUrl = DEFAULT_API_URL,
  model = 'qwen3-asr-flash',
  fetchImpl = globalThis.fetch,
  timeoutMs = 60_000,
} = {}) {
  return {
    async transcribe({ audioDataUri, locale = 'zh-CN' }) {
      if (!apiKey) throw new Error('        DASHSCOPE_API_KEY');
      if (!/^data:audio\/wav;base64,[A-Za-z0-9+/=]+$/i.test(String(audioDataUri ?? ''))) {
```

```
        throw new Error('      Base64 WAV   ');
      }
      const language = languageFromLocale(locale);
      const response = await fetchImpl(apiUrl, {
        method: 'POST',
        headers: {
          Authorization: `Bearer ${apiKey}`,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          model,
          messages: [{
            role: 'user',
            content: [{
              type: 'input_audio',
              input_audio: { data: audioDataUri },
            }],
          }],
          stream: false,
          asr_options: {
            ...(language ? { language } : {}),
            enable_itn: true,
          },
        }),
        signal: AbortSignal.timeout(timeoutMs),
      });

      if (!response.ok) {
        let detail = '';
        try {
          const payload = await response.json();
          detail = payload?.error?.message || payload?.message || '';
        } catch {
          detail = '';
        }
        throw new Error(detail || `         HTTP ${response.status} `);
      }
      const payload = await response.json();
      const transcript = payload?.choices?.[0]?.message?.content;
      if (typeof transcript !== 'string' || !transcript.trim()) {
        throw new Error('            ');
      }
      return transcript.trim();
    },
  };
}
```

server/auth.js

```
import { createHmac, timingSafeEqual } from 'node:crypto';

function sign(encodedPayload, secret) {
  return createHmac('sha256', secret).update(encodedPayload).digest('base64url');
}

export function issueToken(user, secret, ttlMs = 12 * 60 * 60 * 1000) {
  const payload = Buffer.from(JSON.stringify({
    sub: user.id,
    username: user.username,
    displayName: user.displayName,
    role: user.role,
    exp: Date.now() + ttlMs,
  })).toString('base64url');
  return `${payload}.${sign(payload, secret)}`;
}

export function verifyToken(token, secret) {
  try {
    const [payload, signature, extra] = String(token ?? '').split('.');
    if (!payload || !signature || extra) return null;
    const expected = Buffer.from(sign(payload, secret));
    const received = Buffer.from(signature);
    if (expected.length !== received.length || !timingSafeEqual(expected, received)) return null;
    const parsed = JSON.parse(Buffer.from(payload, 'base64url').toString('utf8'));
    if (!parsed.exp || parsed.exp <= Date.now()) return null;
```

```
    return parsed;
  } catch {
    return null;
  }
}
```

server/password.js

```
import { randomBytes, scrypt as scryptCallback, timingSafeEqual } from 'node:crypto';
import { promisify } from 'node:util';

const scrypt = promisify(scryptCallback);

export async function hashPassword(password, salt = randomBytes(16)) {
  const normalizedSalt = Buffer.isBuffer(salt) ? salt : Buffer.from(salt, 'base64');
  const derivedKey = await scrypt(String(password), normalizedSalt, 64);
  return {
    salt: normalizedSalt.toString('base64'),
    hash: Buffer.from(derivedKey).toString('base64'),
  };
}

export async function verifyPassword(password, record) {
  try {
    const expected = Buffer.from(record.hash, 'base64');
    const actual = Buffer.from(await scrypt(
      String(password),
      Buffer.from(record.salt, 'base64'),
      expected.length,
    ));
    return expected.length === actual.length && timingSafeEqual(expected, actual);
  } catch {
    return false;
  }
}
```

apps/shared/app.js

```
import { createSpeechEngine } from './speech-engine.js';
import { createAudioRecorder } from './audio-recorder.js';
import { parseCoachReply } from './coach-reply.js';

const appId = document.body.dataset.app;
for (const link of document.querySelectorAll('[data-imagegen-link]')) {
  link.href = `http://${window.location.hostname}:4050/`;
}
const storageKey = `hsk-conversation:${appId}`;
const sessionKey = `hsk-session:${appId}`;
const speech = createSpeechEngine(window);
const audioRecorder = createAudioRecorder(window);
const elements = {
  loginOverlay: document.querySelector('#login-overlay'),
  loginForm: document.querySelector('#login-form'),
  loginError: document.querySelector('#login-error'),
  username: document.querySelector('#username'),
  password: document.querySelector('#password'),
  userName: document.querySelector('#user-name'),
  logout: document.querySelector('#logout-button'),
  chat: document.querySelector('#chat-log'),
  form: document.querySelector('#message-form'),
  input: document.querySelector('#message-input'),
  send: document.querySelector('#send-button'),
  mic: document.querySelector('#mic-button'),
  continuous: document.querySelector('#continuous-mode'),
  continuousStatus: document.querySelector('#continuous-status'),
  voiceStatus: document.querySelector('#voice-status'),
  voiceName: document.querySelector('#voice-name'),
  scenario: document.querySelector('#scenario'),
  level: document.querySelector('#hsk-level'),
  reset: document.querySelector('#reset-button'),
  duration: document.querySelector('#session-duration'),
  messageCount: document.querySelector('#message-count'),
```

```
  points: document.querySelector('#points'),
  progress: document.querySelector('#mission-progress'),
  currentScenario: document.querySelectorAll('[data-current-scenario]'),
};

const state = {
  token: '',
  user: null,
  messages: [],
  busy: false,
  listening: false,
  continuous: false,
  startedAt: Date.now(),
  restartTimer: null,
  recording: false,
};
const DEMO_LOGIN = {
  student01: { username: 'student01', password: 'Maple-Study-2026-01!' }, student02: { username: 'student02',
password: 'Maple-Study-2026-02!' },
  student03: { username: 'student03', password: 'Maple-Study-2026-03!' }, student04: { username: 'student04',
password: 'Maple-Study-2026-04!' },
  student05: { username: 'student05', password: 'Maple-Study-2026-05!' }, student06: { username: 'student06',
password: 'Maple-Study-2026-06!' },
  student07: { username: 'student07', password: 'Maple-Study-2026-07!' }, student08: { username: 'student08',
password: 'Maple-Study-2026-08!' },
  student09: { username: 'student09', password: 'Maple-Study-2026-09!' }, student10: { username: 'student10',
password: 'Maple-Study-2026-10!' },
  admin01: { username: 'admin01', password: 'Console-Admin-2026-01!' }, admin02: { username: 'admin02', password:
'Console-Admin-2026-02!' },
};

function readStoredJson(key, fallback) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch {
    return fallback;
  }
}

function escapeText(value) {
  const holder = document.createElement('span');
  holder.textContent = value;
  return holder.innerHTML;
}

function formatTime(timestamp = Date.now()) {
  return new Intl.DateTimeFormat('zh-CN', {
    hour: '2-digit',
    minute: '2-digit',
    hour12: false,
  }).format(timestamp);
}

function updateStats() {
  const learnerTurns = state.messages.filter((message) => message.role === 'user').length;
  elements.messageCount.textContent = String(learnerTurns);
  if (elements.points) elements.points.textContent = String(learnerTurns * 15);
  if (elements.progress) {
    const percentage = Math.min(100, learnerTurns * 20);
    elements.progress.style.width = `${percentage}%`;
    elements.progress.parentElement?.setAttribute('aria-valuenow', String(percentage));
  }
}

function saveConversation() {
  localStorage.setItem(storageKey, JSON.stringify({
    messages: state.messages.slice(-80).map(({ role, content }) => ({ role, content })),
    scenario: elements.scenario.value,
    level: Number(elements.level.value),
    updatedAt: Date.now(),
  }));
  updateStats();
}

function addMessage(role, content, { pending = false, persist = true, recording = null } = {}) {
  const emptyState = elements.chat.querySelector('.empty-conversation');
  emptyState?.remove();
  const article = document.createElement('article');
```

```
  article.className = `message message--${role}${pending ? ' message--pending' : ''}`;
  const coachReply = role === 'assistant' && !pending ? parseCoachReply(content) : null;
  const visibleContent = coachReply?.message || content;
  const playback = recording?.url ? `
    <div class="voice-playback">
      <span>      </span>
      <audio controls preload="metadata" src="${escapeText(recording.url)}" aria-label="        "></audio>
    </div>
  ` : '';
  const practice = coachReply ? `
    <div class="coach-practice">
      ${coachReply.shadowing ? `<div><span>   </span><strong>${escapeText(coachReply.shadowing)}</strong></div>` :
''}
      <div><span>    </span><strong>${escapeText(coachReply.requirement)}</strong></div>
    </div>
  ` : '';
  article.innerHTML = `
    <div class="message__meta">
      <span>${role === 'user' ? escapeText(state.user?.displayName || ' ') : 'AI     '}</span>
      <time>${formatTime()}</time>
    </div>
    <p>${escapeText(visibleContent)}</p>
    ${playback}
    ${practice}
  `;
  elements.chat.append(article);
  elements.chat.scrollTop = elements.chat.scrollHeight;
  if (!pending && persist) {
    state.messages.push({ role, content, recording });
    saveConversation();
  }
  return article;
}

function renderHistory() {
  elements.chat.innerHTML = '';
  if (!state.messages.length) {
    const scenario = elements.scenario.options[elements.scenario.selectedIndex]?.text || '    ';
    elements.chat.innerHTML = `
      <div class="empty-conversation">
        <span class="empty-conversation__character" aria-hidden="true"> </span>
        <h2>          </h2>
        <p>     ${escapeText(scenario)}                    </p>
      </div>
    `;
  } else {
    state.messages.forEach((message) => addMessage(message.role, message.content, {
      persist: false,
      recording: message.recording,
    }));
  }
  updateStats();
}

function setVoiceStatus(text, tone = 'ready') {
  elements.voiceStatus.textContent = text;
  elements.voiceStatus.dataset.tone = tone;
}

function setBusy(busy) {
  state.busy = busy;
  elements.send.disabled = busy;
  elements.mic.disabled = busy;
  elements.input.disabled = busy;
  elements.send.textContent = busy ? '    ' : '  ';
}

async function api(path, options = {}) {
  const response = await fetch(path, {
    ...options,
    headers: {
      'content-type': 'application/json',
      ...(state.token ? { authorization: `Bearer ${state.token}` } : {}),
      ...options.headers,
    },
  });
  const body = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(body.error || `     ${response.status} `);
```

```
  return body;
}

async function sendMessage(rawText, { recording = null } = {}) {
  const text = String(rawText || '').trim();
  if (!text || state.busy) return;
  clearTimeout(state.restartTimer);
  elements.input.value = '';
  addMessage('user', text, { recording });
  const pending = addMessage('assistant', '        ...', { pending: true, persist: false });
  setBusy(true);
  setVoiceStatus('AI     ', 'working');
  try {
    const result = await api('/api/chat', {
      method: 'POST',
      body: JSON.stringify({
        level: Number(elements.level.value),
        scenario: elements.scenario.value,
        messages: state.messages,
      }),
    });
    pending.remove();
    addMessage('assistant', result.reply);
    const coachReply = parseCoachReply(result.reply);
    setVoiceStatus('        ', 'speaking');
    const spokenReply = coachReply.shadowing
      ? `${coachReply.message}     ${coachReply.shadowing}`
      : coachReply.message;
    await speech.speak(spokenReply);
    setVoiceStatus('     ', 'ready');
    if (state.continuous) {
      state.restartTimer = setTimeout(() => startListening(), 350);
    }
  } catch (error) {
    pending.remove();
    addMessage('system', error.message, { persist: false });
    setVoiceStatus(error.message, 'error');
  } finally {
    setBusy(false);
    elements.input.focus();
  }
}

async function startListening() {
  if (state.listening || state.busy) return;
  if (!speech.support.recognition) {
    setVoiceStatus('                    Edge   Chrome', 'error');
    return;
  }
  let finalTranscript = '';
  let finalized = false;
  async function finalizeVoiceTurn() {
    if (finalized) return;
    finalized = true;
    let recording = null;
    try {
      recording = await audioRecorder.stop();
    } catch (error) {
      setVoiceStatus(error.message, 'error');
    } finally {
      state.recording = false;
    }
    if (finalTranscript) {
      setVoiceStatus(recording ? '          ' : '         ', 'working');
      await sendMessage(finalTranscript, { recording });
    } else if (recording?.url) {
      audioRecorder.revoke(recording.url);
    }
  }
  try {
    await audioRecorder.start();
    state.recording = true;
    speech.listen({
      onStart() {
        state.listening = true;
        elements.mic.classList.add('is-listening');
        elements.mic.setAttribute('aria-label', '      ');
        setVoiceStatus('        ', 'listening');
```

```
      },
      onInterim(text) {
        elements.input.value = text;
      },
      onFinal(text) {
        finalTranscript = text;
        elements.input.value = text;
      },
      onEnd() {
        state.listening = false;
        elements.mic.classList.remove('is-listening');
        elements.mic.setAttribute('aria-label', '      ');
        void finalizeVoiceTurn();
      },
      onError(error) {
        state.listening = false;
        const messages = {
          'not-allowed': '        ',
          'no-speech': '            ',
          network: '           ',
        };
        setVoiceStatus(messages[error] || `       ${error}`, 'error');
        void finalizeVoiceTurn();
      },
    });
  } catch (error) {
    setVoiceStatus(error.message, 'error');
  }
}

function stopListening() {
  clearTimeout(state.restartTimer);
  speech.stopListening();
  state.listening = false;
  elements.mic.classList.remove('is-listening');
  elements.mic.setAttribute('aria-label', '      ');
  setVoiceStatus('       ', 'ready');
}

function restoreConversation() {
  const saved = readStoredJson(storageKey, {});
  state.messages = Array.isArray(saved.messages) ? saved.messages.slice(-80) : [];
  if (saved.scenario && [...elements.scenario.options].some((option) => option.value === saved.scenario)) {
    elements.scenario.value = saved.scenario;
  }
  if (saved.level) elements.level.value = String(saved.level);
  renderHistory();
  syncScenarioLabels();
}

function syncScenarioLabels() {
  const label = elements.scenario.options[elements.scenario.selectedIndex]?.text || elements.scenario.value;
  elements.currentScenario.forEach((element) => { element.textContent = label; });
}

function completeLogin(session) {
  state.token = session.token;
  state.user = session.user;
  sessionStorage.setItem(sessionKey, JSON.stringify(session));
  elements.userName.textContent = session.user.displayName;
  elements.loginOverlay.hidden = true;
  document.body.classList.add('is-authenticated');
  restoreConversation();
  elements.input.focus();
}

elements.loginForm.addEventListener('submit', async (event) => {
  event.preventDefault();
  elements.loginError.textContent = '';
  const submit = elements.loginForm.querySelector('button[type="submit"]');
  submit.disabled = true;
  try {
    const session = await api('/api/login', {
      method: 'POST',
      body: JSON.stringify({
        username: elements.username.value,
        password: elements.password.value,
      }),
```

```
    });
    completeLogin(session);
  } catch (error) {
    elements.loginError.textContent = error.message;
  } finally {
    submit.disabled = false;
  }
});

document.querySelector('#demo-login')?.addEventListener('click', () => {
  const account = DEMO_LOGIN[document.querySelector('#demo-account')?.value];
  if (!account) return;
  elements.username.value = account.username;
  elements.password.value = account.password;
  elements.loginForm.requestSubmit();
});

elements.form.addEventListener('submit', (event) => {
  event.preventDefault();
  void sendMessage(elements.input.value);
});

elements.mic.addEventListener('click', () => {
  if (state.listening) stopListening();
  else startListening();
});

elements.continuous.addEventListener('change', () => {
  state.continuous = elements.continuous.checked;
  elements.continuousStatus.textContent = state.continuous ? '       ' : '      ';
  if (!state.continuous) clearTimeout(state.restartTimer);
});

elements.scenario.addEventListener('change', () => {
  syncScenarioLabels();
  saveConversation();
  renderHistory();
});

elements.level.addEventListener('change', saveConversation);

elements.reset.addEventListener('click', () => {
  speech.cancel();
  clearTimeout(state.restartTimer);
  state.messages.forEach((message) => audioRecorder.revoke(message.recording?.url));
  state.messages = [];
  state.startedAt = Date.now();
  saveConversation();
  renderHistory();
  setVoiceStatus('      ', 'ready');
});

elements.logout.addEventListener('click', () => {
  speech.cancel();
  sessionStorage.removeItem(sessionKey);
  state.token = '';
  state.user = null;
  document.body.classList.remove('is-authenticated');
  elements.loginOverlay.hidden = false;
  elements.password.value = '';
  elements.username.focus();
});

setInterval(() => {
  const elapsed = Math.max(0, Date.now() - state.startedAt);
  const minutes = Math.floor(elapsed / 60_000).toString().padStart(2, '0');
  const seconds = Math.floor((elapsed % 60_000) / 1000).toString().padStart(2, '0');
  elements.duration.textContent = `${minutes}:${seconds}`;
}, 1000);

elements.voiceName.textContent = speech.getVoiceName();
if (!speech.support.recognition || !speech.support.synthesis) {
  setVoiceStatus('                Edge   Chrome', 'error');
}

try {
  const existingSession = JSON.parse(sessionStorage.getItem(sessionKey));
  if (existingSession?.token && existingSession?.user) completeLogin(existingSession);
```

```
} catch {
  sessionStorage.removeItem(sessionKey);
}
```

apps/shared/audio-recorder.js

```
const MIME_CANDIDATES = [
  'audio/webm;codecs=opus',
  'audio/webm',
  'audio/mp4',
];

export function createAudioRecorder(windowObject = globalThis.window) {
  const MediaRecorderClass = windowObject?.MediaRecorder;
  const mediaDevices = windowObject?.navigator?.mediaDevices;
  const urlApi = windowObject?.URL;
  let recorder = null;
  let stream = null;
  let chunks = [];
  let mimeType = '';

  function stopTracks() {
    stream?.getTracks?.().forEach((track) => track.stop());
    stream = null;
  }

  return {
    supported: Boolean(MediaRecorderClass && mediaDevices?.getUserMedia && urlApi?.createObjectURL),

    async start() {
      if (!MediaRecorderClass || !mediaDevices?.getUserMedia || !urlApi?.createObjectURL) {
        throw new Error('            ');
      }
      if (recorder?.state === 'recording') return;
      stream = await mediaDevices.getUserMedia({
        audio: {
          channelCount: 1,
          echoCancellation: true,
          noiseSuppression: true,
        },
      });
      mimeType = MIME_CANDIDATES.find((candidate) => (
        typeof MediaRecorderClass.isTypeSupported !== 'function'
        || MediaRecorderClass.isTypeSupported(candidate)
      )) || '';
      chunks = [];
      recorder = new MediaRecorderClass(stream, mimeType ? { mimeType } : undefined);
      recorder.ondataavailable = (event) => {
        if (event.data?.size) chunks.push(event.data);
      };
      recorder.start();
    },

    stop() {
      if (!recorder || recorder.state === 'inactive') {
        stopTracks();
        return Promise.resolve(null);
      }
      return new Promise((resolve, reject) => {
        recorder.onerror = (event) => {
          stopTracks();
          reject(new Error(event?.error?.message || '      '));
        };
        recorder.onstop = () => {
          const blob = new Blob(chunks, { type: mimeType || chunks[0]?.type || 'audio/webm' });
          stopTracks();
          recorder = null;
          chunks = [];
          if (!blob.size) {
            resolve(null);
            return;
          }
          resolve({
            blob,
            mimeType: mimeType || blob.type,
```

```
            url: urlApi.createObjectURL(blob),
          });
        };
        recorder.stop();
      });
    },

    revoke(url) {
      if (url) urlApi?.revokeObjectURL?.(url);
    },
  };
}
```

apps/shared/speech-engine.js

```
export function selectChineseVoice(voices = []) {
  const chineseVoices = voices.filter((voice) => /^zh(?:-|_)/i.test(voice.lang || ''));
  if (!chineseVoices.length) return voices.find((voice) => voice.default) || voices[0] || null;
  return chineseVoices.find((voice) => /Microsoft|Xiaoxiao|Yunxi|Xiaoyi/i.test(voice.name || ''))
    || chineseVoices.find((voice) => voice.default)
    || chineseVoices[0];
}

function collectTranscript(event) {
  let interim = '';
  let finalText = '';
  for (let index = event.resultIndex ?? 0; index < event.results.length; index += 1) {
    const result = event.results[index];
    const text = result?.[0]?.transcript?.trim() || '';
    if (result?.isFinal) finalText += text;
    else interim += text;
  }
  return { finalText, interim };
}

export function createSpeechEngine(windowObject = globalThis.window) {
  const Recognition = windowObject?.SpeechRecognition || windowObject?.webkitSpeechRecognition;
  const synthesis = windowObject?.speechSynthesis;
  const Utterance = windowObject?.SpeechSynthesisUtterance;
  let recognition = null;

  return {
    support: {
      recognition: Boolean(Recognition),
      synthesis: Boolean(synthesis && Utterance),
    },

    listen({ onFinal, onInterim, onStart, onEnd, onError } = {}) {
      if (!Recognition) throw new Error('                    Edge   Chrome');
      recognition?.abort?.();
      recognition = new Recognition();
      recognition.lang = 'zh-CN';
      recognition.interimResults = true;
      recognition.continuous = false;
      recognition.maxAlternatives = 1;
      recognition.onstart = () => onStart?.();
      recognition.onresult = (event) => {
        const { finalText, interim } = collectTranscript(event);
        if (interim) onInterim?.(interim);
        if (finalText) onFinal?.(finalText);
      };
      recognition.onerror = (event) => onError?.(event?.error || 'unknown');
      recognition.onend = () => onEnd?.();
      recognition.start();
      return recognition;
    },

    stopListening() {
      recognition?.stop?.();
    },

    cancel() {
      recognition?.abort?.();
      synthesis?.cancel?.();
    },
```

```
    async speak(text, { rate = 0.92, pitch = 1 } = {}) {
      if (!synthesis || !Utterance) {
        throw new Error('             ');
      }
      synthesis.cancel();
      const utterance = new Utterance(String(text));
      utterance.lang = 'zh-CN';
      utterance.rate = rate;
      utterance.pitch = pitch;
      utterance.voice = selectChineseVoice(synthesis.getVoices());
      return new Promise((resolve, reject) => {
        utterance.onend = resolve;
        utterance.onerror = (event) => reject(new Error(event?.error || '      '));
        synthesis.speak(utterance);
      });
    },

    getVoiceName() {
      return selectChineseVoice(synthesis?.getVoices?.() || [])?.name || '         ';
    },
  };
}
```

apps/platform/platform.js

```
const $ = (id) => document.getElementById(id);
const DEMO = {
  student01: { username: 'student01', password: 'Maple-Study-2026-01!' }, student02: { username: 'student02',
password: 'Maple-Study-2026-02!' },
  student03: { username: 'student03', password: 'Maple-Study-2026-03!' }, student04: { username: 'student04',
password: 'Maple-Study-2026-04!' },
  student05: { username: 'student05', password: 'Maple-Study-2026-05!' }, student06: { username: 'student06',
password: 'Maple-Study-2026-06!' },
  student07: { username: 'student07', password: 'Maple-Study-2026-07!' }, student08: { username: 'student08',
password: 'Maple-Study-2026-08!' },
  student09: { username: 'student09', password: 'Maple-Study-2026-09!' }, student10: { username: 'student10',
password: 'Maple-Study-2026-10!' },
  admin01: { username: 'admin01', password: 'Console-Admin-2026-01!' }, admin02: { username: 'admin02', password:
'Console-Admin-2026-02!' },
};
let user = null;
let questions = [];
let attempts = new Set();
let roundCorrect = 0;

function escapeHtml(value) {
  return String(value ?? '').replace(/[&<>"']/g, (character) => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;',
  })[character]);
}

async function api(path, options = {}) {
  const response = await fetch(path, {
    ...options,
    headers: {
      'content-type': 'application/json',
      ...(user?.token ? { authorization: `Bearer ${user.token}` } : {}),
      ...options.headers,
    },
  });
  const body = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(body.error || `     ${response.status} `);
  return body;
}

function speak(text) {
  if (!text || !('speechSynthesis' in window)) return;
  speechSynthesis.cancel();
  const utterance = new SpeechSynthesisUtterance(text);
  utterance.lang = 'zh-CN';
  utterance.rate = 0.9;
  speechSynthesis.speak(utterance);
}

function login(session) {
```

```
  user = session;
  sessionStorage.setItem('hsk-platform-user', JSON.stringify(user));
  $('login').hidden = true;
  $('app').hidden = false;
  $('account-name').textContent = `${user.displayName} · ${user.role === 'teacher' ? '   ' : '  '}`;
  $('admin-nav').hidden = user.role !== 'teacher';
  void render();
  void renderStats();
}

async function authenticate(username, password) {
  $('login-error').textContent = '';
  try {
    const result = await api('/api/login', {
      method: 'POST',
      body: JSON.stringify({ username, password, appId: 'hsk-platform' }),
    });
    login({ ...result.user, token: result.token });
  } catch (error) {
    $('login-error').textContent = error.message;
  }
}

async function render() {
  const level = Number($('level').value);
  const section = $('section').value;
  const params = new URLSearchParams({ level: String(level), section });
  try {
    const result = await api(`/api/practice/questions?${params}`);
    questions = result.questions;
    $('source-links').innerHTML = result.sources.map((source) =>
      `<a href="${escapeHtml(source.url)}" target="_blank" rel="noopener noreferrer">${escapeHtml(source.label)}</
a>`,
    ).join('');
    attempts = new Set();
    roundCorrect = 0;
    $('score').textContent = `0 / ${questions.length}`;
    $('score-bar').className = 'score-bar--0';
    $('question-list').innerHTML = questions.length ? questions.map((question, index) => `
      <article class="question" data-id="${escapeHtml(question.id)}">
        <h2>${index + 1}. ${escapeHtml(question.question)}</h2>
        ${question.audioText ? `<button class="speak" data-speak="${escapeHtml(question.audioText)}">      </
button>` : ''}
        <div class="options">${question.options.map((option, optionIndex) => `<button data-answer="${optionIndex +
1}">${optionIndex + 1}. ${escapeHtml(option)}</button>`).join('')}</div>
        <div class="explain" hidden></div>
        <small class="muted">${escapeHtml(question.source || 'HSK         ')}</small>
      </article>`).join('') : '<div class="question"><h2>          </h2><p class="muted">           </p></div>';
    bindQuestions();
  } catch (error) {
    $('question-list').textContent = error.message;
  }
}

function bindQuestions() {
  document.querySelectorAll('[data-speak]').forEach((button) => {
    button.onclick = () => speak(button.dataset.speak);
  });
  document.querySelectorAll('.question').forEach((card) => {
    card.querySelectorAll('[data-answer]').forEach((button) => {
      button.onclick = async () => {
        const question = questions.find((item) => item.id === card.dataset.id);
        if (!question || attempts.has(question.id)) return;
        attempts.add(question.id);
        card.querySelectorAll('[data-answer]').forEach((option) => { option.disabled = true; });
        try {
          const result = await api('/api/practice/attempts', {
            method: 'POST',
            body: JSON.stringify({ questionId: question.id, level: question.level, section: question.section,
answer: Number(button.dataset.answer) }),
          });
          button.classList.add(result.correct ? 'correct' : 'wrong');
          if (!result.correct) card.querySelector(`[data-answer="${result.correctAnswer}"]`)?.classList.add(
'correct');
          card.querySelector('.explain').textContent = result.explanation;
          card.querySelector('.explain').hidden = false;
          if (result.correct) roundCorrect += 1;
          $('score').textContent = `${roundCorrect} / ${questions.length}`;
```

```
          const percentage = questions.length ? Math.round(roundCorrect / questions.length * 100 / 25) * 25 : 0;
          $('score-bar').className = `score-bar--${percentage}`;
          await renderStats();
        } catch (error) {
          attempts.delete(question.id);
          card.querySelectorAll('[data-answer]').forEach((option) => { option.disabled = false; });
          card.querySelector('.explain').textContent = error.message;
          card.querySelector('.explain').hidden = false;
        }
      };
    });
  });
}

async function renderStats() {
  try {
    const stats = await api('/api/practice/stats');
    $('stat-total').textContent = stats.total;
    $('stat-rate').textContent = `${stats.total ? Math.round(stats.correct / stats.total * 100) : 0}%`;
    $('chart').innerHTML = ['listening', 'reading', 'speaking'].map((section, index) => {
      const label = { listening: '  ', reading: '  ', speaking: '  ' }[section];
      const data = stats.bySection[section] || { total: 0, rate: 0 };
      const tier = data.total ? Math.min(4, Math.max(1, Math.ceil(data.rate / 25))) : 0;
      return `<div class="chart__bar chart__bar--${tier}" aria-label="${label}    ${Math.round(data.
rate)}%"><span>${label} ${Math.round(data.rate)}% · ${data.total} </span></div>`;
    }).join('');
  } catch (error) {
    $('stat-total').textContent = '—';
    $('stat-rate').textContent = '—';
    $('chart').textContent = error.message;
  }
}

async function renderPublished() {
  if (user?.role !== 'teacher') return;
  const params = new URLSearchParams({ level: $('new-level').value, section: $('new-section').value });
  try {
    const result = await api(`/api/practice/questions?${params}`);
    $('published').innerHTML = result.questions.filter((question) => !String(question.id).startsWith('sample-')).
map((question) =>
      `<article><b>HSK ${question.level} · ${escapeHtml(question.section)}</b><br>${escapeHtml(question.
question)}</article>`,
    ).join('');
  } catch (error) {
    $('published').textContent = error.message;
  }
}

$('demo-login').onclick = () => {
  const account = DEMO[$('demo-account').value];
  if (account) void authenticate(account.username, account.password);
};
$('manual-login').onsubmit = (event) => {
  event.preventDefault();
  void authenticate($('login-user').value, $('login-pass').value);
};
$('logout').onclick = () => {
  sessionStorage.removeItem('hsk-platform-user');
  location.reload();
};
document.querySelectorAll('nav [data-view]').forEach((button) => {
  button.onclick = () => {
    document.querySelectorAll('.view').forEach((view) => { view.hidden = view.id !== button.dataset.view; });
    document.querySelectorAll('nav button').forEach((item) => item.classList.toggle('active', item === button));
    if (button.dataset.view === 'stats') void renderStats();
    if (button.dataset.view === 'admin') void renderPublished();
  };
});
$('level').onchange = () => void render();
$('section').onchange = () => void render();
$('new-level').onchange = () => void renderPublished();
$('new-section').onchange = () => void renderPublished();
$('speak-tip').onclick = () => speak('               ');
$('publish-form').onsubmit = async (event) => {
  event.preventDefault();
  const form = event.currentTarget;
  const submit = form.querySelector('[type="submit"]');
  submit.disabled = true;
```

```
  try {
    await api('/api/admin/questions', {
      method: 'POST',
      body: JSON.stringify({
        level: Number($('new-level').value), section: $('new-section').value,
        question: $('new-question').value,
        options: $('new-options').value.split('|').map((option) => option.trim()),
        answer: Number($('new-answer').value),
        explanation: $('new-explanation').value,
        audioText: $('new-audio').value,
      }),
    });
    form.reset();
    await renderPublished();
    await render();
  } catch (error) {
    $('publish-error').textContent = error.message;
  } finally {
    submit.disabled = false;
  }
};

try {
  const session = JSON.parse(sessionStorage.getItem('hsk-platform-user'));
  if (session?.token) login(session);
} catch {
  sessionStorage.removeItem('hsk-platform-user');
}
```

android_apps/hsk_dialogue_studio/lib/app.dart

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter/services.dart';

import 'app_config.dart';
import 'coach_api.dart';
import 'native_speech_service.dart';
import 'reply_parser.dart';

const _ink = Color(0xFF1C2420);
const _paper = Color(0xFFF4F6F3);
const _line = Color(0xFFDDE3DE);

class HskVoiceApp extends StatelessWidget {
  const HskVoiceApp({
    super.key,
    required this.config,
    this.coachGateway,
    this.speechGateway,
  });

  final HskAppConfig config;
  final CoachGateway? coachGateway;
  final SpeechGateway? speechGateway;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: config.title,
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme:
            ColorScheme.fromSeed(
              seedColor: config.primary,
              surface: _paper,
            ).copyWith(
              primary: config.primary,
              secondary: config.secondary,
              outlineVariant: _line,
            ),
        scaffoldBackgroundColor: _paper,
```

```
        cardTheme: const CardThemeData(
          elevation: 0,
          margin: EdgeInsets.zero,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.all(Radius.circular(8)),
            side: BorderSide(color: _line),
          ),
        ),
        inputDecorationTheme: const InputDecorationTheme(
          filled: true,
          fillColor: Colors.white,
          border: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(8)),
            borderSide: BorderSide(color: _line),
          ),
          enabledBorder: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(8)),
            borderSide: BorderSide(color: _line),
          ),
        ),
      ),
      home: _AppRoot(
        config: config,
        coach:
            coachGateway ??
            CoachApi(
              baseUrl: config.apiBaseUrl,
              clientApp: config.applicationId,
            ),
        speech: speechGateway ?? NativeSpeechService(),
      ),
    );
  }
}

class _AppRoot extends StatefulWidget {
  const _AppRoot({
    required this.config,
    required this.coach,
    required this.speech,
  });

  final HskAppConfig config;
  final CoachGateway coach;
  final SpeechGateway speech;

  @override
  State<_AppRoot> createState() => _AppRootState();
}

class _AppRootState extends State<_AppRoot> {
  LoginSession? _session;

  @override
  Widget build(BuildContext context) {
    final session = _session;
    if (session == null) {
      return _LoginPage(
        config: widget.config,
        coach: widget.coach,
        onLoggedIn: (value) => setState(() => _session = value),
      );
    }
    return _ConversationPage(
      config: widget.config,
      coach: widget.coach,
      speech: widget.speech,
      session: session,
      onLogout: () => setState(() => _session = null),
    );
  }
}

class _LoginPage extends StatefulWidget {
  const _LoginPage({
    required this.config,
    required this.coach,
    required this.onLoggedIn,
```

```
  });

  final HskAppConfig config;
  final CoachGateway coach;
  final ValueChanged<LoginSession> onLoggedIn;

  @override
  State<_LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<_LoginPage> {
  final _username = TextEditingController();
  final _password = TextEditingController();
  bool _loading = false;
  String? _error;
  @override
  void dispose() {
    _username.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    if (_loading) return;
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      final session = await widget.coach.login(
        _username.text.trim(),
        _password.text,
      );
      if (mounted) widget.onLoggedIn(session);
    } on CoachApiException catch (error) {
      if (mounted) setState(() => _error = error.message);
    } catch (error) {
      if (mounted) setState(() => _error = '     $error');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _loginWithSso() async {
    if (_loading) return;
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      final session = await widget.coach.loginWithOidc(
        issuer: widget.config.oidcIssuer,
        clientId: widget.config.oidcClientId,
        redirectUrl: widget.config.oidcRedirectUrl,
      );
      if (mounted) widget.onLoggedIn(session);
    } on CoachApiException catch (error) {
      if (mounted) setState(() => _error = error.message);
    } catch (error) {
      if (mounted) setState(() => _error = '       $error');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: ConstrainedBox(
              constraints: const BoxConstraints(maxWidth: 430),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: <Widget>[
                  Align(
```

```
            alignment: Alignment.centerLeft,
            child: Container(
              width: 54,
              height: 54,
              alignment: Alignment.center,
              decoration: BoxDecoration(
                color: widget.config.primary,
                borderRadius: BorderRadius.circular(8),
              ),
              child: const Text(
                ' ',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 25,
                  fontWeight: FontWeight.w800,
                ),
              ),
            ),
          ),
          const SizedBox(height: 24),
          Text(
            widget.config.title,
            style: Theme.of(context).textTheme.headlineMedium?.copyWith(
              color: _ink,
              fontWeight: FontWeight.w800,
            ),
          ),
          const SizedBox(height: 6),
          Text(widget.config.subtitle),
          const SizedBox(height: 30),
          FilledButton.icon(
            onPressed: _loading ? null : _loginWithSso,
            icon: const Icon(Icons.key),
            label: const Text('        '),
          ),
          const SizedBox(height: 18),
          const Divider(),
          const SizedBox(height: 12),
          TextField(
            controller: _username,
            autofillHints: const <String>[AutofillHints.username],
            decoration: const InputDecoration(
              labelText: '  ',
              prefixIcon: Icon(Icons.person_outline),
            ),
          ),
          const SizedBox(height: 14),
          TextField(
            controller: _password,
            obscureText: true,
            autofillHints: const <String>[AutofillHints.password],
            onSubmitted: (_) => _login(),
            decoration: const InputDecoration(
              labelText: '  ',
              prefixIcon: Icon(Icons.lock_outline),
            ),
          ),
          if (_error != null) ...<Widget>[
            const SizedBox(height: 12),
            _Notice(text: _error!, isError: true),
          ],
          const SizedBox(height: 18),
          FilledButton.icon(
            onPressed: _loading ? null : _login,
            icon: _loading
                ? const SizedBox(
                    width: 18,
                    height: 18,
                    child: CircularProgressIndicator(
                      strokeWidth: 2,
                      color: Colors.white,
                    ),
                  )
                : const Icon(Icons.login),
            label: const Text('      '),
          ),
        ],
      ),
```

```
            ),
          ),
        ),
      ),
    );
  }
}

class _ConversationPage extends StatefulWidget {
  const _ConversationPage({
    required this.config,
    required this.coach,
    required this.speech,
    required this.session,
    required this.onLogout,
  });

  final HskAppConfig config;
  final CoachGateway coach;
  final SpeechGateway speech;
  final LoginSession session;
  final VoidCallback onLogout;

  @override
  State<_ConversationPage> createState() => _ConversationPageState();
}

class _ConversationPageState extends State<_ConversationPage> {
  final _input = TextEditingController();
  final _scroll = ScrollController();
  late final List<ChatMessage> _messages;
  int _level = 3;
  int _scenarioIndex = 0;
  bool _recording = false;
  bool _sending = false;
  bool _continuous = false;
  bool _hasEarlierHistory = false;
  bool _loadingEarlierHistory = false;
  String _status = '     ';
  String? _error;
  ConversationCursor? _historyCursor;

  String get _scenario => widget.config.scenarios[_scenarioIndex];

  @override
  void initState() {
    super.initState();
    _messages = <ChatMessage>[
      const ChatMessage(
        role: 'assistant',
        content: '               \n               \n                   ',
      ),
    ];
    unawaited(_restoreHistory());
  }

  Future<void> _restoreHistory() async {
    try {
      final history = await widget.coach.loadLatestConversation(
        token: widget.session.token,
      );
      if (!mounted || history.messages.isEmpty) return;
      setState(() {
        _messages
          ..clear()
          ..addAll(history.messages);
        _historyCursor = history.nextCursor;
        _hasEarlierHistory = history.hasMore && history.nextCursor != null;
        if (history.level != null) _level = history.level!;
        final scenarioIndex = widget.config.scenarios.indexOf(
          history.scenario ?? '',
        );
        if (scenarioIndex >= 0) _scenarioIndex = scenarioIndex;
        _status = '         ';
      });
      _scrollToBottom();
    } on CoachApiException {
      if (mounted) setState(() => _status = '     ');
```

```
  }
}

@override
void dispose() {
  _input.dispose();
  _scroll.dispose();
  if (_recording) widget.speech.cancelRecording();
  widget.speech.stopSpeaking();
  widget.speech.stopRecordingPlayback();
  super.dispose();
}

Future<void> _startRecording() async {
  if (_recording || _sending) return;
  setState(() {
    _error = null;
    _status = '                ';
  });
  try {
    await widget.speech.stopSpeaking();
    await widget.speech.stopRecordingPlayback();
    await widget.speech.startRecording();
    if (mounted) setState(() => _recording = true);
  } on PlatformException catch (error) {
    if (mounted) {
      setState(() {
        _recording = false;
        _error = error.message ?? '          ';
        _status = '      ';
      });
    }
  }
}

Future<void> _finishRecording() async {
  if (!_recording) return;
  RecordedAudio? recording;
  setState(() {
    _recording = false;
    _sending = true;
    _status = '        ';
    _error = null;
  });
  try {
    recording = await widget.speech.stopRecording();
    final transcript = await widget.coach.transcribe(
      token: widget.session.token,
      wavBytes: recording.bytes,
      level: _level,
      scenario: _scenario,
    );
    if (!mounted) return;
    setState(() => _sending = false);
    await _sendUserMessage(transcript, recordingPath: recording.path);
  } on PlatformException catch (error) {
    if (mounted) {
      setState(() {
        _sending = false;
        _error = error.message ?? '       ';
        _status = '     ';
      });
    }
  } on CoachApiException catch (error) {
    if (mounted) {
      setState(() {
        _sending = false;
        if (recording != null) {
          _messages.add(
            ChatMessage(
              role: 'user',
              content: '             ',
              recordingPath: recording.path,
            ),
          );
        }
        _error = error.message;
        _status = '               ';
```

```
      });
      _scrollToBottom();
    }
  }
}

Future<void> _sendText() async {
  final text = _input.text.trim();
  if (text.isEmpty) return;
  _input.clear();
  await _sendUserMessage(text);
}

Future<void> _sendUserMessage(String text, {String? recordingPath}) async {
  if (_sending) return;
  setState(() {
    _sending = true;
    _error = null;
    _status = 'AI     ';
    _messages.add(
      ChatMessage(role: 'user', content: text, recordingPath: recordingPath),
    );
  });
  _scrollToBottom();
  try {
    final reply = await widget.coach.chat(
      token: widget.session.token,
      level: _level,
      scenario: _scenario,
      messages: _messages,
    );
    if (!mounted) return;
    setState(() {
      _messages.add(ChatMessage(role: 'assistant', content: reply));
      _status = '         ';
    });
    _scrollToBottom();
    final parsed = parseCoachReply(reply);
    final spoken = parsed.shadowing.isEmpty
        ? parsed.message
        : '${parsed.message}     ${parsed.shadowing}';
    await widget.speech.speak(spoken);
    if (!mounted) return;
    setState(() {
      _sending = false;
      _status = '     ';
    });
    if (_continuous) await _startRecording();
  } on CoachApiException catch (error) {
    if (mounted) {
      setState(() {
        _error = error.message;
        _status = '      ';
      });
    }
  } on PlatformException catch (error) {
    if (mounted) {
      setState(() {
        _error = error.message ?? '       ';
        _status = '       ';
      });
    }
  } finally {
    if (mounted) setState(() => _sending = false);
    _scrollToBottom();
  }
}

void _scrollToBottom() {
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!_scroll.hasClients) return;
    _scroll.animateTo(
      _scroll.position.maxScrollExtent,
      duration: const Duration(milliseconds: 240),
      curve: Curves.easeOut,
    );
  });
}
```

```
  bool _handleHistoryScroll(UserScrollNotification notification) {
    if (notification.direction == ScrollDirection.forward &&
        notification.metrics.extentBefore <= 48) {
      unawaited(_loadEarlierHistory());
    }
    return false;
  }

  Future<void> _loadEarlierHistory() async {
    final cursor = _historyCursor;
    if (_loadingEarlierHistory || !_hasEarlierHistory || cursor == null) return;

    final oldMaxExtent = _scroll.hasClients
        ? _scroll.position.maxScrollExtent
        : 0.0;
    final oldPixels = _scroll.hasClients ? _scroll.position.pixels : 0.0;
    var insertedMessages = false;
    setState(() => _loadingEarlierHistory = true);
    try {
      final history = await widget.coach.loadLatestConversation(
        token: widget.session.token,
        before: cursor,
      );
      if (!mounted) return;
      if (history.messages.isEmpty) {
        setState(() {
          _hasEarlierHistory = false;
          _historyCursor = null;
        });
        return;
      }
      setState(() {
        _messages.insertAll(0, history.messages);
        _historyCursor = history.nextCursor;
        _hasEarlierHistory = history.hasMore && history.nextCursor != null;
      });
      insertedMessages = true;
    } on CoachApiException catch (error) {
      if (mounted) {
        setState(() {
          _error = error.message;
          _status = '      ';
        });
      }
    } finally {
      if (mounted) {
        setState(() => _loadingEarlierHistory = false);
        await WidgetsBinding.instance.endOfFrame;
        if (insertedMessages && _scroll.hasClients) {
          final delta = _scroll.position.maxScrollExtent - oldMaxExtent;
          _scroll.jumpTo(
            (oldPixels + delta)
                .clamp(
                  _scroll.position.minScrollExtent,
                  _scroll.position.maxScrollExtent,
                )
                .toDouble(),
          );
        }
      }
    }
  }

  Future<void> _editServer() async {
    final controller = TextEditingController(text: widget.coach.baseUrl);
    final value = await showDialog<String>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('      '),
        content: TextField(
          controller: controller,
          keyboardType: TextInputType.url,
        ),
        actions: <Widget>[
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('  '),
          ),
```

```
          FilledButton(
            onPressed: () => Navigator.pop(context, controller.text.trim()),
            child: const Text('  '),
          ),
        ],
      ),
    );
    controller.dispose();
    if (value != null && value.isNotEmpty) widget.coach.baseUrl = value;
  }

  void _newConversation() {
    widget.coach.resetConversation();
    setState(() {
      _messages
        ..clear()
        ..add(
          const ChatMessage(
            role: 'assistant',
            content: '        \n           \n                ',
          ),
        );
      _status = '      ';
      _error = null;
    });
  }

  @override
  Widget build(BuildContext context) {
    final learnerTurns = _messages.where((item) => item.role == 'user').length;
    return Scaffold(
      appBar: AppBar(
        titleSpacing: 16,
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(
              widget.config.title,
              style: const TextStyle(fontSize: 17, fontWeight: FontWeight.w800),
            ),
            Text(
              widget.session.user.displayName,
              style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w400),
            ),
          ],
        ),
        actions: <Widget>[
          IconButton(
            tooltip: '     ',
            onPressed: _newConversation,
            icon: const Icon(Icons.refresh),
          ),
          IconButton(
            tooltip: '    ',
            onPressed: _editServer,
            icon: const Icon(Icons.settings_outlined),
          ),
          IconButton(
            tooltip: '    ',
            onPressed: widget.onLogout,
            icon: const Icon(Icons.logout),
          ),
        ],
      ),
      body: SafeArea(
        top: false,
        child: Column(
          children: <Widget>[
            Container(
              padding: const EdgeInsets.fromLTRB(12, 10, 12, 12),
              decoration: const BoxDecoration(
                color: Colors.white,
                border: Border(bottom: BorderSide(color: _line)),
              ),
              child: Column(
                children: <Widget>[
                  Row(
                    children: <Widget>[
```

```
            Expanded(
              child: DropdownButtonFormField<int>(
                isExpanded: true,
                initialValue: _level,
                decoration: const InputDecoration(
                  labelText: 'HSK   ',
                  isDense: true,
                ),
                items: List<DropdownMenuItem<int>>.generate(
                  6,
                  (index) => DropdownMenuItem<int>(
                    value: index + 1,
                    child: Text('HSK ${index + 1}'),
                  ),
                ),
                onChanged: (value) {
                  if (value != null) setState(() => _level = value);
                },
              ),
            ),
            const SizedBox(width: 10),
            Expanded(
              child: DropdownButtonFormField<int>(
                isExpanded: true,
                initialValue: _scenarioIndex,
                decoration: const InputDecoration(
                  labelText: '    ',
                  isDense: true,
                ),
                items: widget.config.scenarios
                    .asMap()
                    .entries
                    .map(
                      (entry) => DropdownMenuItem<int>(
                        value: entry.key,
                        child: Text(
                          entry.value,
                          overflow: TextOverflow.ellipsis,
                        ),
                      ),
                    )
                    .toList(),
                onChanged: (value) {
                  if (value != null) {
                    setState(() => _scenarioIndex = value);
                  }
                },
              ),
            ),
          ],
        ),
        if (widget.config.isMission) ...<Widget>[
          const SizedBox(height: 10),
          Row(
            children: <Widget>[
              Text('     $learnerTurns / 5'),
              const SizedBox(width: 10),
              Expanded(
                child: LinearProgressIndicator(
                  value: (learnerTurns / 5).clamp(0, 1),
                ),
              ),
            ],
          ),
        ],
      ],
    ),
  ),
  Expanded(
    child: NotificationListener<UserScrollNotification>(
      onNotification: _handleHistoryScroll,
      child: ListView.builder(
        controller: _scroll,
        padding: const EdgeInsets.fromLTRB(14, 16, 14, 12),
        itemCount:
            _messages.length +
            (_hasEarlierHistory || _loadingEarlierHistory ? 1 : 0),
        itemBuilder: (context, index) {
```

```
          final hasHeader =
              _hasEarlierHistory || _loadingEarlierHistory;
          if (hasHeader && index == 0) {
            return SizedBox(
              height: _loadingEarlierHistory ? 40 : 1,
              child: _loadingEarlierHistory
                  ? const Center(child: CircularProgressIndicator())
                  : null,
            );
          }
          final messageIndex = index - (hasHeader ? 1 : 0);
          return _MessageBubble(
            message: _messages[messageIndex],
            primary: widget.config.primary,
            secondary: widget.config.secondary,
            speech: widget.speech,
          );
        },
      ),
    ),
  ),
  if (_error != null)
    Padding(
      padding: const EdgeInsets.fromLTRB(12, 0, 12, 8),
      child: _Notice(text: _error!, isError: true),
    ),
  Container(
    padding: EdgeInsets.fromLTRB(
      12,
      9,
      12,
      MediaQuery.viewInsetsOf(context).bottom > 0 ? 8 : 12,
    ),
    decoration: const BoxDecoration(
      color: Colors.white,
      border: Border(top: BorderSide(color: _line)),
    ),
    child: Column(
      children: <Widget>[
        Row(
          children: <Widget>[
            Icon(
              _recording ? Icons.graphic_eq : Icons.circle,
              size: _recording ? 18 : 8,
              color: _recording ? Colors.red : Colors.green,
            ),
            const SizedBox(width: 7),
            Expanded(
              child: Text(
                _status,
                style: const TextStyle(fontSize: 11),
              ),
            ),
            Text(
              '$learnerTurns  ',
              style: const TextStyle(fontSize: 11),
            ),
          ],
        ),
        const SizedBox(height: 8),
        Row(
          crossAxisAlignment: CrossAxisAlignment.end,
          children: <Widget>[
            IconButton.filled(
              tooltip: _recording ? '     ' : '      ',
              style: IconButton.styleFrom(
                backgroundColor: _recording
                    ? Colors.red
                    : widget.config.primary,
              ),
              onPressed: _sending
                  ? null
                  : (_recording ? _finishRecording : _startRecording),
              icon: Icon(_recording ? Icons.stop : Icons.mic),
            ),
            const SizedBox(width: 8),
            Expanded(
              child: TextField(
```

```
                        controller: _input,
                        enabled: !_sending && !_recording,
                        minLines: 1,
                        maxLines: 3,
                        textInputAction: TextInputAction.send,
                        onSubmitted: (_) => _sendText(),
                        decoration: const InputDecoration(
                          hintText: '    ',
                          isDense: true,
                        ),
                      ),
                    ),
                    const SizedBox(width: 8),
                    IconButton.filled(
                      tooltip: '    ',
                      onPressed: _sending || _recording ? null : _sendText,
                      icon: _sending
                          ? const SizedBox(
                              width: 18,
                              height: 18,
                              child: CircularProgressIndicator(
                                strokeWidth: 2,
                                color: Colors.white,
                              ),
                            )
                          : const Icon(Icons.send),
                    ),
                  ],
                ),
                Row(
                  children: <Widget>[
                    Switch(
                      value: _continuous,
                      onChanged: (value) {
                        setState(() => _continuous = value);
                      },
                    ),
                    const Text('      ', style: TextStyle(fontSize: 12)),
                    const Spacer(),
                    Text(_scenario, style: const TextStyle(fontSize: 11)),
                  ],
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  );
 }
}

class _MessageBubble extends StatelessWidget {
  const _MessageBubble({
    required this.message,
    required this.primary,
    required this.secondary,
    required this.speech,
  });

  final ChatMessage message;
  final Color primary;
  final Color secondary;
  final SpeechGateway speech;

  @override
  Widget build(BuildContext context) {
    final isUser = message.role == 'user';
    final parsed = isUser ? null : parseCoachReply(message.content);
    return Align(
      alignment: isUser ? Alignment.centerRight : Alignment.centerLeft,
      child: Container(
        constraints: const BoxConstraints(maxWidth: 360),
        margin: const EdgeInsets.only(bottom: 12),
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: isUser ? primary : Colors.white,
          border: isUser ? null : Border.all(color: _line),
```

```
          borderRadius: BorderRadius.circular(8),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(
              parsed?.message ?? message.content,
              style: TextStyle(
                color: isUser ? Colors.white : _ink,
                height: 1.5,
              ),
            ),
            if (message.recordingPath != null) ...<Widget>[
              const SizedBox(height: 8),
              _RecordingPlaybackButton(
                path: message.recordingPath!,
                speech: speech,
                foreground: Colors.white,
              ),
            ],
            if (parsed != null) ...<Widget>[
              const SizedBox(height: 9),
              Container(
                width: double.infinity,
                padding: const EdgeInsets.all(10),
                decoration: BoxDecoration(
                  color: secondary.withValues(alpha: 0.09),
                  border: Border(left: BorderSide(color: secondary, width: 3)),
                ),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: <Widget>[
                    if (parsed.shadowing.isNotEmpty) ...<Widget>[
                      const Text(
                        '   ',
                        style: TextStyle(
                          fontSize: 10,
                          fontWeight: FontWeight.w800,
                        ),
                      ),
                      const SizedBox(height: 3),
                      Text(parsed.shadowing),
                      Align(
                        alignment: Alignment.centerRight,
                        child: IconButton(
                          tooltip: '      ',
                          visualDensity: VisualDensity.compact,
                          onPressed: () => speech.speak(parsed.shadowing),
                          icon: Icon(Icons.volume_up_outlined, color: primary),
                        ),
                      ),
                    ],
                    const Text(
                      '    ',
                      style: TextStyle(
                        fontSize: 10,
                        fontWeight: FontWeight.w800,
                      ),
                    ),
                    const SizedBox(height: 3),
                    Text(
                      parsed.requirement,
                      style: const TextStyle(fontSize: 12),
                    ),
                  ],
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

class _RecordingPlaybackButton extends StatefulWidget {
  const _RecordingPlaybackButton({
    required this.path,
```

```
    required this.speech,
    required this.foreground,
  });

  final String path;
  final SpeechGateway speech;
  final Color foreground;

  @override
  State<_RecordingPlaybackButton> createState() =>
      _RecordingPlaybackButtonState();
}

class _RecordingPlaybackButtonState extends State<_RecordingPlaybackButton> {
  bool _playing = false;
  bool _loading = false;
  int _generation = 0;

  Future<void> _toggle() async {
    final generation = ++_generation;
    if (_playing) {
      await widget.speech.stopRecordingPlayback();
      if (mounted) setState(() => _playing = false);
      return;
    }
    setState(() => _loading = true);
    try {
      final duration = await widget.speech.playRecording(widget.path);
      if (!mounted) return;
      setState(() {
        _loading = false;
        _playing = true;
      });
      await Future<void>.delayed(duration + const Duration(milliseconds: 200));
      if (mounted && generation == _generation) {
        setState(() => _playing = false);
      }
    } on PlatformException catch (error) {
      if (!mounted) return;
      setState(() {
        _loading = false;
        _playing = false;
      });
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(error.message ?? '       ')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return TextButton.icon(
      onPressed: _loading ? null : _toggle,
      style: TextButton.styleFrom(foregroundColor: widget.foreground),
      icon: _loading
          ? SizedBox(
              width: 16,
              height: 16,
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: widget.foreground,
              ),
            )
          : Icon(
              _playing ? Icons.stop_circle_outlined : Icons.play_circle_outline,
            ),
      label: Text(_playing ? '    ' : '      '),
    );
  }
}

class _Notice extends StatelessWidget {
  const _Notice({required this.text, required this.isError});

  final String text;
  final bool isError;

  @override
```

```
  Widget build(BuildContext context) {
    final color = isError ? const Color(0xFFAA3038) : Colors.green;
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(10),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.08),
        borderRadius: BorderRadius.circular(8),
        border: Border.all(color: color.withValues(alpha: 0.25)),
      ),
      child: Row(
        children: <Widget>[
          Icon(
            isError ? Icons.error_outline : Icons.info_outline,
            color: color,
          ),
          const SizedBox(width: 8),
          Expanded(
            child: Text(text, style: TextStyle(color: color)),
          ),
        ],
      ),
    );
  }
}
```

android_apps/hsk_dialogue_studio/lib/coach_api.dart

```
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter_appauth/flutter_appauth.dart';

class SessionUser {
  const SessionUser({required this.displayName, required this.username});

  final String displayName;
  final String username;
}

class LoginSession {
  const LoginSession({required this.token, required this.user});

  final String token;
  final SessionUser user;
}

class ChatMessage {
  const ChatMessage({
    required this.role,
    required this.content,
    this.recordingPath,
    this.source = 'text',
  });

  final String role;
  final String content;
  final String? recordingPath;
  final String source;

  Map<String, String> toJson() => <String, String>{
    'role': role,
    'content': content,
    'source': recordingPath == null ? source : 'voice',
  };
}

class ConversationHistory {
  const ConversationHistory({
    required this.messages,
    this.level,
    this.scenario,
```

```
    this.hasMore = false,
    this.nextCursor,
  });

  final List<ChatMessage> messages;
  final int? level;
  final String? scenario;
  final bool hasMore;
  final ConversationCursor? nextCursor;
}

class ConversationCursor {
  const ConversationCursor({
    required this.conversationId,
    required this.id,
    required this.createdAt,
  });

  final String conversationId;
  final String id;
  final String createdAt;
}

abstract class CoachGateway {
  String get baseUrl;
  set baseUrl(String value);

  Future<LoginSession> login(String username, String password);

  Future<LoginSession> loginWithOidc({
    required String issuer,
    required String clientId,
    required String redirectUrl,
  });

  Future<String> transcribe({
    required String token,
    required List<int> wavBytes,
    required int level,
    required String scenario,
  });

  Future<String> chat({
    required String token,
    required int level,
    required String scenario,
    required List<ChatMessage> messages,
  });

  Future<ConversationHistory> loadLatestConversation({
    required String token,
    ConversationCursor? before,
  });

  void resetConversation();
}

class CoachApi implements CoachGateway {
  CoachApi({
    this.baseUrl = '***',
    this.clientApp = 'unknown',
  });

  @override
  String baseUrl;
  final String clientApp;
  final FlutterAppAuth _appAuth = const FlutterAppAuth();
  String? _conversationId;

  @override
  Future<LoginSession> login(String username, String password) async {
    final payload = await _post('/api/login', <String, Object?>{
      'username': username,
      'password': password,
      'appId': clientApp,
    });
    final user = payload['user'] as Map<String, dynamic>?;
    final token = payload['token'] as String?;
```

```
    if (user == null || token == null || token.isEmpty) {
      throw const CoachApiException('           ');
    }
    _conversationId = null;
    return LoginSession(
      token: token,
      user: SessionUser(
        displayName: user['displayName'] as String? ?? username,
        username: user['username'] as String? ?? username,
      ),
    );
  }

  @override
  Future<LoginSession> loginWithOidc({
    required String issuer,
    required String clientId,
    required String redirectUrl,
  }) async {
    final result = await _appAuth.authorizeAndExchangeCode(
      AuthorizationTokenRequest(
        clientId,
        redirectUrl,
        issuer: issuer,
        scopes: const <String>['openid', 'profile'],
      ),
    );
    final accessToken = result.accessToken;
    if (accessToken == null || accessToken.isEmpty) {
      throw const CoachApiException('            ');
    }
    final payload = await _post('/api/auth/oidc', <String, Object?>{
      'accessToken': accessToken,
      'appId': clientApp,
    });
    final user = payload['user'] as Map<String, dynamic>?;
    final token = payload['token'] as String?;
    if (user == null || token == null || token.isEmpty) {
      throw const CoachApiException('             ');
    }
    _conversationId = null;
    return LoginSession(
      token: token,
      user: SessionUser(
        displayName: user['displayName'] as String? ?? '  ',
        username: user['username'] as String? ?? '',
      ),
    );
  }

  @override
  Future<String> transcribe({
    required String token,
    required List<int> wavBytes,
    required int level,
    required String scenario,
  }) async {
    final payload = await _post(
      '/api/transcribe',
      <String, Object?>{
        'audio': 'data:audio/wav;base64,${base64Encode(wavBytes)}',
        'locale': 'zh-CN',
        'appId': clientApp,
        'conversationId': _conversationId,
        'level': level,
        'scenario': scenario,
      },
      token: token,
      timeout: const Duration(seconds: 65),
    );
    final transcript = payload['transcript'] as String?;
    if (transcript == null || transcript.trim().isEmpty) {
      throw const CoachApiException('               ');
    }
    _conversationId = payload['conversationId'] as String? ?? _conversationId;
    return transcript.trim();
  }
  @override
```

```
  Future<String> chat({
    required String token,
    required int level,
    required String scenario,
    required List<ChatMessage> messages,
  }) async {
    final payload = await _post(
      '/api/chat',
      <String, Object?>{
        'level': level,
        'scenario': scenario,
        'appId': clientApp,
        'conversationId': _conversationId,
        'messages': messages
            .skip(messages.length > 24 ? messages.length - 24 : 0)
            .map((message) => message.toJson())
            .toList(),
      },
      token: token,
      timeout: const Duration(seconds: 45),
    );
    final reply = payload['reply'] as String?;
    if (reply == null || reply.trim().isEmpty) {
      throw const CoachApiException('AI          ');
    }
    _conversationId = payload['conversationId'] as String? ?? _conversationId;
    return reply.trim();
  }

  @override
  Future<ConversationHistory> loadLatestConversation({
    required String token,
    ConversationCursor? before,
  }) async {
    final query = <String, String>{'appId': clientApp, 'limit': '80'};
    if (before != null) {
      query.addAll(<String, String>{
        'cursorConversationId': before.conversationId,
        'cursorId': before.id,
        'cursorCreatedAt': before.createdAt,
      });
    }
    final payload = await _get(
      '/api/history/latest?${Uri(queryParameters: query).query}',
      token: token,
    );
    _conversationId = payload['conversationId'] as String?;
    final rawMessages = payload['messages'] as List<dynamic>? ?? const [];
    final rawCursor = payload['nextCursor'] as Map<String, dynamic>?;
    final cursor = rawCursor == null
        ? null
        : ConversationCursor(
            conversationId: rawCursor['conversationId'] as String? ?? '',
            id: rawCursor['id']?.toString() ?? '',
            createdAt: rawCursor['createdAt'] as String? ?? '',
          );
    return ConversationHistory(
      level: payload['level'] as int?,
      scenario: payload['scenario'] as String?,
      hasMore: payload['hasMore'] == true && cursor != null,
      nextCursor: cursor,
      messages: rawMessages
          .whereType<Map<String, dynamic>>()
          .map(
            (message) => ChatMessage(
              role: message['role'] as String? ?? 'assistant',
              content: message['content'] as String? ?? '',
              source: message['source'] as String? ?? 'text',
            ),
          )
          .where((message) => message.content.isNotEmpty)
          .toList(),
    );
  }

  @override
  void resetConversation() {
    _conversationId = null;
```

```
  }

  Future<Map<String, dynamic>> _get(
    String path, {
    required String token,
  }) async {
    final client = HttpClient()..connectionTimeout = const Duration(seconds: 8);
    try {
      final root = baseUrl.trim().replaceAll(RegExp(r'/+$'), '');
      final request = await client.getUrl(Uri.parse('$root$path'));
      request.headers.set(HttpHeaders.authorizationHeader, 'Bearer $token');
      final response = await request.close().timeout(
        const Duration(seconds: 15),
      );
      final payload =
          jsonDecode(await utf8.decoder.bind(response).join())
              as Map<String, dynamic>;
      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw CoachApiException(payload['error'] as String? ?? '         ');
      }
      return payload;
    } on SocketException {
      throw const CoachApiException('         ');
    } on TimeoutException {
      throw const CoachApiException('         ');
    } finally {
      client.close(force: true);
    }
  }

  Future<Map<String, dynamic>> _post(
    String path,
    Map<String, Object?> body, {
    String? token,
    Duration timeout = const Duration(seconds: 15),
  }) async {
    final client = HttpClient()..connectionTimeout = const Duration(seconds: 8);
    try {
      final root = baseUrl.trim().replaceAll(RegExp(r'/+$'), '');
      final request = await client.postUrl(Uri.parse('$root$path'));
      request.headers.contentType = ContentType.json;
      if (token != null) {
        request.headers.set(HttpHeaders.authorizationHeader, 'Bearer $token');
      }
      request.write(jsonEncode(body));
      final response = await request.close().timeout(timeout);
      final responseBody = await utf8.decoder.bind(response).join();
      final payload = jsonDecode(responseBody) as Map<String, dynamic>;
      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw CoachApiException(
          payload['error'] as String? ?? '       ${response.statusCode}  ',
        );
      }
      return payload;
    } on SocketException {
      throw const CoachApiException(
        '         USB         adb reverse tcp:8040 tcp:8040 ',
      );
    } on TimeoutException {
      throw const CoachApiException('             ');
    } on FormatException {
      throw const CoachApiException('         ');
    } finally {
      client.close(force: true);
    }
  }
}

class CoachApiException implements Exception {
  const CoachApiException(this.message);
  final String message;
  @override
  String toString() => message;
}
```

android_apps/hsk_mission_lab/lib/app.dart

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter/services.dart';

import 'app_config.dart';
import 'coach_api.dart';
import 'native_speech_service.dart';
import 'reply_parser.dart';

const _ink = Color(0xFF1C2420);
const _paper = Color(0xFFF4F6F3);
const _line = Color(0xFFDDE3DE);

class HskVoiceApp extends StatelessWidget {
  const HskVoiceApp({
    super.key,
    required this.config,
    this.coachGateway,
    this.speechGateway,
  });

  final HskAppConfig config;
  final CoachGateway? coachGateway;
  final SpeechGateway? speechGateway;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: config.title,
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme:
            ColorScheme.fromSeed(
              seedColor: config.primary,
              surface: _paper,
            ).copyWith(
              primary: config.primary,
              secondary: config.secondary,
              outlineVariant: _line,
            ),
        scaffoldBackgroundColor: _paper,
        cardTheme: const CardThemeData(
          elevation: 0,
          margin: EdgeInsets.zero,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.all(Radius.circular(8)),
            side: BorderSide(color: _line),
          ),
        ),
        inputDecorationTheme: const InputDecorationTheme(
          filled: true,
          fillColor: Colors.white,
          border: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(8)),
            borderSide: BorderSide(color: _line),
          ),
          enabledBorder: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(8)),
            borderSide: BorderSide(color: _line),
          ),
        ),
      ),
      home: _AppRoot(
        config: config,
        coach:
            coachGateway ??
            CoachApi(
              baseUrl: config.apiBaseUrl,
              clientApp: config.applicationId,
            ),
        speech: speechGateway ?? NativeSpeechService(),
```

```
      ),
    );
  }
}

class _AppRoot extends StatefulWidget {
  const _AppRoot({
    required this.config,
    required this.coach,
    required this.speech,
  });

  final HskAppConfig config;
  final CoachGateway coach;
  final SpeechGateway speech;

  @override
  State<_AppRoot> createState() => _AppRootState();
}

class _AppRootState extends State<_AppRoot> {
  LoginSession? _session;

  @override
  Widget build(BuildContext context) {
    final session = _session;
    if (session == null) {
      return _LoginPage(
        config: widget.config,
        coach: widget.coach,
        onLoggedIn: (value) => setState(() => _session = value),
      );
    }
    return _ConversationPage(
      config: widget.config,
      coach: widget.coach,
      speech: widget.speech,
      session: session,
      onLogout: () => setState(() => _session = null),
    );
  }
}

class _LoginPage extends StatefulWidget {
  const _LoginPage({
    required this.config,
    required this.coach,
    required this.onLoggedIn,
  });

  final HskAppConfig config;
  final CoachGateway coach;
  final ValueChanged<LoginSession> onLoggedIn;

  @override
  State<_LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<_LoginPage> {
  final _username = TextEditingController();
  final _password = TextEditingController();
  bool _loading = false;
  String? _error;
  @override
  void dispose() {
    _username.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    if (_loading) return;
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      final session = await widget.coach.login(
```

```
        _username.text.trim(),
        _password.text,
      );
      if (mounted) widget.onLoggedIn(session);
    } on CoachApiException catch (error) {
      if (mounted) setState(() => _error = error.message);
    } catch (error) {
      if (mounted) setState(() => _error = '     $error');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _loginWithSso() async {
    if (_loading) return;
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      final session = await widget.coach.loginWithOidc(
        issuer: widget.config.oidcIssuer,
        clientId: widget.config.oidcClientId,
        redirectUrl: widget.config.oidcRedirectUrl,
      );
      if (mounted) widget.onLoggedIn(session);
    } on CoachApiException catch (error) {
      if (mounted) setState(() => _error = error.message);
    } catch (error) {
      if (mounted) setState(() => _error = '       $error');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: ConstrainedBox(
              constraints: const BoxConstraints(maxWidth: 430),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: <Widget>[
                  Align(
                    alignment: Alignment.centerLeft,
                    child: Container(
                      width: 54,
                      height: 54,
                      alignment: Alignment.center,
                      decoration: BoxDecoration(
                        color: widget.config.primary,
                        borderRadius: BorderRadius.circular(8),
                      ),
                      child: const Text(
                        ' ',
                        style: TextStyle(
                          color: Colors.white,
                          fontSize: 25,
                          fontWeight: FontWeight.w800,
                        ),
                      ),
                    ),
                  ),
                ),
                const SizedBox(height: 24),
                Text(
                  widget.config.title,
                  style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                    color: _ink,
                    fontWeight: FontWeight.w800,
                  ),
                ),
                const SizedBox(height: 6),
                Text(widget.config.subtitle),
                const SizedBox(height: 30),
```

```
                  FilledButton.icon(
                    onPressed: _loading ? null : _loginWithSso,
                    icon: const Icon(Icons.key),
                    label: const Text('        '),
                  ),
                  const SizedBox(height: 18),
                  const Divider(),
                  const SizedBox(height: 12),
                  TextField(
                    controller: _username,
                    autofillHints: const <String>[AutofillHints.username],
                    decoration: const InputDecoration(
                      labelText: '  ',
                      prefixIcon: Icon(Icons.person_outline),
                    ),
                  ),
                  const SizedBox(height: 14),
                  TextField(
                    controller: _password,
                    obscureText: true,
                    autofillHints: const <String>[AutofillHints.password],
                    onSubmitted: (_) => _login(),
                    decoration: const InputDecoration(
                      labelText: '  ',
                      prefixIcon: Icon(Icons.lock_outline),
                    ),
                  ),
                  if (_error != null) ...<Widget>[
                    const SizedBox(height: 12),
                    _Notice(text: _error!, isError: true),
                  ],
                  const SizedBox(height: 18),
                  FilledButton.icon(
                    onPressed: _loading ? null : _login,
                    icon: _loading
                        ? const SizedBox(
                            width: 18,
                            height: 18,
                            child: CircularProgressIndicator(
                              strokeWidth: 2,
                              color: Colors.white,
                            ),
                          )
                        : const Icon(Icons.login),
                    label: const Text('      '),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class _ConversationPage extends StatefulWidget {
  const _ConversationPage({
    required this.config,
    required this.coach,
    required this.speech,
    required this.session,
    required this.onLogout,
  });

  final HskAppConfig config;
  final CoachGateway coach;
  final SpeechGateway speech;
  final LoginSession session;
  final VoidCallback onLogout;

  @override
  State<_ConversationPage> createState() => _ConversationPageState();
}

class _ConversationPageState extends State<_ConversationPage> {
  final _input = TextEditingController();
  final _scroll = ScrollController();
```

```
  late final List<ChatMessage> _messages;
  int _level = 3;
  int _scenarioIndex = 0;
  bool _recording = false;
  bool _sending = false;
  bool _continuous = false;
  bool _hasEarlierHistory = false;
  bool _loadingEarlierHistory = false;
  String _status = '     ';
  String? _error;
  ConversationCursor? _historyCursor;

  String get _scenario => widget.config.scenarios[_scenarioIndex];

  @override
  void initState() {
    super.initState();
    _messages = <ChatMessage>[
      const ChatMessage(
        role: 'assistant',
        content: '               \n               \n                   ',
      ),
    ];
    unawaited(_restoreHistory());
  }

  Future<void> _restoreHistory() async {
    try {
      final history = await widget.coach.loadLatestConversation(
        token: widget.session.token,
      );
      if (!mounted || history.messages.isEmpty) return;
      setState(() {
        _messages
          ..clear()
          ..addAll(history.messages);
        _historyCursor = history.nextCursor;
        _hasEarlierHistory = history.hasMore && history.nextCursor != null;
        if (history.level != null) _level = history.level!;
        final scenarioIndex = widget.config.scenarios.indexOf(
          history.scenario ?? '',
        );
        if (scenarioIndex >= 0) _scenarioIndex = scenarioIndex;
        _status = '         ';
      });
      _scrollToBottom();
    } on CoachApiException {
      if (mounted) setState(() => _status = '     ');
    }
  }

  @override
  void dispose() {
    _input.dispose();
    _scroll.dispose();
    if (_recording) widget.speech.cancelRecording();
    widget.speech.stopSpeaking();
    widget.speech.stopRecordingPlayback();
    super.dispose();
  }

  Future<void> _startRecording() async {
    if (_recording || _sending) return;
    setState(() {
      _error = null;
      _status = '                ';
    });
    try {
      await widget.speech.stopSpeaking();
      await widget.speech.stopRecordingPlayback();
      await widget.speech.startRecording();
      if (mounted) setState(() => _recording = true);
    } on PlatformException catch (error) {
      if (mounted) {
        setState(() {
          _recording = false;
          _error = error.message ?? '          ';
          _status = '      ';
```

```
        });
      }
    }
  }

  Future<void> _finishRecording() async {
    if (!_recording) return;
    RecordedAudio? recording;
    setState(() {
      _recording = false;
      _sending = true;
      _status = '        ';
      _error = null;
    });
    try {
      recording = await widget.speech.stopRecording();
      final transcript = await widget.coach.transcribe(
        token: widget.session.token,
        wavBytes: recording.bytes,
        level: _level,
        scenario: _scenario,
      );
      if (!mounted) return;
      setState(() => _sending = false);
      await _sendUserMessage(transcript, recordingPath: recording.path);
    } on PlatformException catch (error) {
      if (mounted) {
        setState(() {
          _sending = false;
          _error = error.message ?? '       ';
          _status = '     ';
        });
      }
    } on CoachApiException catch (error) {
      if (mounted) {
        setState(() {
          _sending = false;
          if (recording != null) {
            _messages.add(
              ChatMessage(
                role: 'user',
                content: '             ',
                recordingPath: recording.path,
              ),
            );
          }
          _error = error.message;
          _status = '               ';
        });
        _scrollToBottom();
      }
    }
  }

  Future<void> _sendText() async {
    final text = _input.text.trim();
    if (text.isEmpty) return;
    _input.clear();
    await _sendUserMessage(text);
  }

  Future<void> _sendUserMessage(String text, {String? recordingPath}) async {
    if (_sending) return;
    setState(() {
      _sending = true;
      _error = null;
      _status = 'AI     ';
      _messages.add(
        ChatMessage(role: 'user', content: text, recordingPath: recordingPath),
      );
    });
    _scrollToBottom();
    try {
      final reply = await widget.coach.chat(
        token: widget.session.token,
        level: _level,
        scenario: _scenario,
        messages: _messages,
```

```
      );
      if (!mounted) return;
      setState(() {
        _messages.add(ChatMessage(role: 'assistant', content: reply));
        _status = '        ';
      });
      _scrollToBottom();
      final parsed = parseCoachReply(reply);
      final spoken = parsed.shadowing.isEmpty
          ? parsed.message
          : '${parsed.message}     ${parsed.shadowing}';
      await widget.speech.speak(spoken);
      if (!mounted) return;
      setState(() {
        _sending = false;
        _status = '     ';
      });
      if (_continuous) await _startRecording();
    } on CoachApiException catch (error) {
      if (mounted) {
        setState(() {
          _error = error.message;
          _status = '      ';
        });
      }
    } on PlatformException catch (error) {
      if (mounted) {
        setState(() {
          _error = error.message ?? '       ';
          _status = '       ';
        });
      }
    } finally {
      if (mounted) setState(() => _sending = false);
      _scrollToBottom();
    }
  }

  void _scrollToBottom() {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!_scroll.hasClients) return;
      _scroll.animateTo(
        _scroll.position.maxScrollExtent,
        duration: const Duration(milliseconds: 240),
        curve: Curves.easeOut,
      );
    });
  }

  bool _handleHistoryScroll(UserScrollNotification notification) {
    if (notification.direction == ScrollDirection.forward &&
        notification.metrics.extentBefore <= 48) {
      unawaited(_loadEarlierHistory());
    }
    return false;
  }

  Future<void> _loadEarlierHistory() async {
    final cursor = _historyCursor;
    if (_loadingEarlierHistory || !_hasEarlierHistory || cursor == null) return;

    final oldMaxExtent = _scroll.hasClients
        ? _scroll.position.maxScrollExtent
        : 0.0;
    final oldPixels = _scroll.hasClients ? _scroll.position.pixels : 0.0;
    var insertedMessages = false;
    setState(() => _loadingEarlierHistory = true);
    try {
      final history = await widget.coach.loadLatestConversation(
        token: widget.session.token,
        before: cursor,
      );
      if (!mounted) return;
      if (history.messages.isEmpty) {
        setState(() {
          _hasEarlierHistory = false;
          _historyCursor = null;
        });
```

```
        return;
      }
      setState(() {
        _messages.insertAll(0, history.messages);
        _historyCursor = history.nextCursor;
        _hasEarlierHistory = history.hasMore && history.nextCursor != null;
      });
      insertedMessages = true;
    } on CoachApiException catch (error) {
      if (mounted) {
        setState(() {
          _error = error.message;
          _status = '      ';
        });
      }
    } finally {
      if (mounted) {
        setState(() => _loadingEarlierHistory = false);
        await WidgetsBinding.instance.endOfFrame;
        if (insertedMessages && _scroll.hasClients) {
          final delta = _scroll.position.maxScrollExtent - oldMaxExtent;
          _scroll.jumpTo(
            (oldPixels + delta)
                .clamp(
                  _scroll.position.minScrollExtent,
                  _scroll.position.maxScrollExtent,
                )
                .toDouble(),
          );
        }
      }
    }
  }

  Future<void> _editServer() async {
    final controller = TextEditingController(text: widget.coach.baseUrl);
    final value = await showDialog<String>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('      '),
        content: TextField(
          controller: controller,
          keyboardType: TextInputType.url,
        ),
        actions: <Widget>[
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('  '),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(context, controller.text.trim()),
            child: const Text('  '),
          ),
        ],
      ),
    );
    controller.dispose();
    if (value != null && value.isNotEmpty) widget.coach.baseUrl = value;
  }

  void _newConversation() {
    widget.coach.resetConversation();
    setState(() {
      _messages
        ..clear()
        ..add(
          const ChatMessage(
            role: 'assistant',
            content: '        \n           \n                ',
          ),
        );
      _status = '      ';
      _error = null;
    });
  }

  @override
  Widget build(BuildContext context) {
```

```
      final learnerTurns = _messages.where((item) => item.role == 'user').length;
      return Scaffold(
        appBar: AppBar(
          titleSpacing: 16,
          title: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: <Widget>[
              Text(
                widget.config.title,
                style: const TextStyle(fontSize: 17, fontWeight: FontWeight.w800),
              ),
              Text(
                widget.session.user.displayName,
                style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w400),
              ),
            ],
          ),
          actions: <Widget>[
            IconButton(
              tooltip: '     ',
              onPressed: _newConversation,
              icon: const Icon(Icons.refresh),
            ),
            IconButton(
              tooltip: '    ',
              onPressed: _editServer,
              icon: const Icon(Icons.settings_outlined),
            ),
            IconButton(
              tooltip: '    ',
              onPressed: widget.onLogout,
              icon: const Icon(Icons.logout),
            ),
          ],
        ),
        body: SafeArea(
          top: false,
          child: Column(
            children: <Widget>[
              Container(
                padding: const EdgeInsets.fromLTRB(12, 10, 12, 12),
                decoration: const BoxDecoration(
                  color: Colors.white,
                  border: Border(bottom: BorderSide(color: _line)),
                ),
                child: Column(
                  children: <Widget>[
                    Row(
                      children: <Widget>[
                        Expanded(
                          child: DropdownButtonFormField<int>(
                            isExpanded: true,
                            initialValue: _level,
                            decoration: const InputDecoration(
                              labelText: 'HSK   ',
                              isDense: true,
                            ),
                            items: List<DropdownMenuItem<int>>.generate(
                              6,
                              (index) => DropdownMenuItem<int>(
                                value: index + 1,
                                child: Text('HSK ${index + 1}'),
                              ),
                            ),
                            onChanged: (value) {
                              if (value != null) setState(() => _level = value);
                            },
                          ),
                        ),
                        const SizedBox(width: 10),
                        Expanded(
                          child: DropdownButtonFormField<int>(
                            isExpanded: true,
                            initialValue: _scenarioIndex,
                            decoration: const InputDecoration(
                              labelText: '    ',
                              isDense: true,
                            ),
```

```
                items: widget.config.scenarios
                    .asMap()
                    .entries
                    .map(
                      (entry) => DropdownMenuItem<int>(
                        value: entry.key,
                        child: Text(
                          entry.value,
                          overflow: TextOverflow.ellipsis,
                        ),
                      ),
                    )
                    .toList(),
                onChanged: (value) {
                  if (value != null) {
                    setState(() => _scenarioIndex = value);
                  }
                },
              ),
            ),
          ],
        ),
        if (widget.config.isMission) ...<Widget>[
          const SizedBox(height: 10),
          Row(
            children: <Widget>[
              Text('     $learnerTurns / 5'),
              const SizedBox(width: 10),
              Expanded(
                child: LinearProgressIndicator(
                  value: (learnerTurns / 5).clamp(0, 1),
                ),
              ),
            ],
          ),
        ],
      ],
    ),
  ),
),
Expanded(
  child: NotificationListener<UserScrollNotification>(
    onNotification: _handleHistoryScroll,
    child: ListView.builder(
      controller: _scroll,
      padding: const EdgeInsets.fromLTRB(14, 16, 14, 12),
      itemCount:
          _messages.length +
          (_hasEarlierHistory || _loadingEarlierHistory ? 1 : 0),
      itemBuilder: (context, index) {
        final hasHeader =
            _hasEarlierHistory || _loadingEarlierHistory;
        if (hasHeader && index == 0) {
          return SizedBox(
            height: _loadingEarlierHistory ? 40 : 1,
            child: _loadingEarlierHistory
                ? const Center(child: CircularProgressIndicator())
                : null,
          );
        }
        final messageIndex = index - (hasHeader ? 1 : 0);
        return _MessageBubble(
          message: _messages[messageIndex],
          primary: widget.config.primary,
          secondary: widget.config.secondary,
          speech: widget.speech,
        );
      },
    ),
  ),
),
if (_error != null)
  Padding(
    padding: const EdgeInsets.fromLTRB(12, 0, 12, 8),
    child: _Notice(text: _error!, isError: true),
  ),
Container(
  padding: EdgeInsets.fromLTRB(
    12,
```

```
  9,
  12,
  MediaQuery.viewInsetsOf(context).bottom > 0 ? 8 : 12,
),
decoration: const BoxDecoration(
  color: Colors.white,
  border: Border(top: BorderSide(color: _line)),
),
child: Column(
  children: <Widget>[
    Row(
      children: <Widget>[
        Icon(
          _recording ? Icons.graphic_eq : Icons.circle,
          size: _recording ? 18 : 8,
          color: _recording ? Colors.red : Colors.green,
        ),
        const SizedBox(width: 7),
        Expanded(
          child: Text(
            _status,
            style: const TextStyle(fontSize: 11),
          ),
        ),
        Text(
          '$learnerTurns  ',
          style: const TextStyle(fontSize: 11),
        ),
      ],
    ),
    const SizedBox(height: 8),
    Row(
      crossAxisAlignment: CrossAxisAlignment.end,
      children: <Widget>[
        IconButton.filled(
          tooltip: _recording ? '     ' : '      ',
          style: IconButton.styleFrom(
            backgroundColor: _recording
                ? Colors.red
                : widget.config.primary,
          ),
          onPressed: _sending
              ? null
              : (_recording ? _finishRecording : _startRecording),
          icon: Icon(_recording ? Icons.stop : Icons.mic),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: TextField(
            controller: _input,
            enabled: !_sending && !_recording,
            minLines: 1,
            maxLines: 3,
            textInputAction: TextInputAction.send,
            onSubmitted: (_) => _sendText(),
            decoration: const InputDecoration(
              hintText: '    ',
              isDense: true,
            ),
          ),
        ),
        const SizedBox(width: 8),
        IconButton.filled(
          tooltip: '    ',
          onPressed: _sending || _recording ? null : _sendText,
          icon: _sending
              ? const SizedBox(
                  width: 18,
                  height: 18,
                  child: CircularProgressIndicator(
                    strokeWidth: 2,
                    color: Colors.white,
                  ),
                )
              : const Icon(Icons.send),
        ),
      ],
    ),
```

```
                  Row(
                    children: <Widget>[
                      Switch(
                        value: _continuous,
                        onChanged: (value) {
                          setState(() => _continuous = value);
                        },
                      ),
                      const Text('      ', style: TextStyle(fontSize: 12)),
                      const Spacer(),
                      Text(_scenario, style: const TextStyle(fontSize: 11)),
                    ],
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _MessageBubble extends StatelessWidget {
  const _MessageBubble({
    required this.message,
    required this.primary,
    required this.secondary,
    required this.speech,
  });

  final ChatMessage message;
  final Color primary;
  final Color secondary;
  final SpeechGateway speech;

  @override
  Widget build(BuildContext context) {
    final isUser = message.role == 'user';
    final parsed = isUser ? null : parseCoachReply(message.content);
    return Align(
      alignment: isUser ? Alignment.centerRight : Alignment.centerLeft,
      child: Container(
        constraints: const BoxConstraints(maxWidth: 360),
        margin: const EdgeInsets.only(bottom: 12),
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: isUser ? primary : Colors.white,
          border: isUser ? null : Border.all(color: _line),
          borderRadius: BorderRadius.circular(8),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(
              parsed?.message ?? message.content,
              style: TextStyle(
                color: isUser ? Colors.white : _ink,
                height: 1.5,
              ),
            ),
            if (message.recordingPath != null) ...<Widget>[
              const SizedBox(height: 8),
              _RecordingPlaybackButton(
                path: message.recordingPath!,
                speech: speech,
                foreground: Colors.white,
              ),
            ],
            if (parsed != null) ...<Widget>[
              const SizedBox(height: 9),
              Container(
                width: double.infinity,
                padding: const EdgeInsets.all(10),
                decoration: BoxDecoration(
                  color: secondary.withValues(alpha: 0.09),
                  border: Border(left: BorderSide(color: secondary, width: 3)),
                ),
```

```
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: <Widget>[
                    if (parsed.shadowing.isNotEmpty) ...<Widget>[
                      const Text(
                        '   ',
                        style: TextStyle(
                          fontSize: 10,
                          fontWeight: FontWeight.w800,
                        ),
                      ),
                      const SizedBox(height: 3),
                      Text(parsed.shadowing),
                      Align(
                        alignment: Alignment.centerRight,
                        child: IconButton(
                          tooltip: '      ',
                          visualDensity: VisualDensity.compact,
                          onPressed: () => speech.speak(parsed.shadowing),
                          icon: Icon(Icons.volume_up_outlined, color: primary),
                        ),
                      ),
                    ],
                    const Text(
                      '    ',
                      style: TextStyle(
                        fontSize: 10,
                        fontWeight: FontWeight.w800,
                      ),
                    ),
                    const SizedBox(height: 3),
                    Text(
                      parsed.requirement,
                      style: const TextStyle(fontSize: 12),
                    ),
                  ],
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

class _RecordingPlaybackButton extends StatefulWidget {
  const _RecordingPlaybackButton({
    required this.path,
    required this.speech,
    required this.foreground,
  });

  final String path;
  final SpeechGateway speech;
  final Color foreground;

  @override
  State<_RecordingPlaybackButton> createState() =>
      _RecordingPlaybackButtonState();
}

class _RecordingPlaybackButtonState extends State<_RecordingPlaybackButton> {
  bool _playing = false;
  bool _loading = false;
  int _generation = 0;

  Future<void> _toggle() async {
    final generation = ++_generation;
    if (_playing) {
      await widget.speech.stopRecordingPlayback();
      if (mounted) setState(() => _playing = false);
      return;
    }
    setState(() => _loading = true);
    try {
      final duration = await widget.speech.playRecording(widget.path);
      if (!mounted) return;
```

```
      setState(() {
        _loading = false;
        _playing = true;
      });
      await Future<void>.delayed(duration + const Duration(milliseconds: 200));
      if (mounted && generation == _generation) {
        setState(() => _playing = false);
      }
    } on PlatformException catch (error) {
      if (!mounted) return;
      setState(() {
        _loading = false;
        _playing = false;
      });
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(error.message ?? '       ')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return TextButton.icon(
      onPressed: _loading ? null : _toggle,
      style: TextButton.styleFrom(foregroundColor: widget.foreground),
      icon: _loading
          ? SizedBox(
              width: 16,
              height: 16,
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: widget.foreground,
              ),
            )
          : Icon(
              _playing ? Icons.stop_circle_outlined : Icons.play_circle_outline,
            ),
      label: Text(_playing ? '    ' : '      '),
    );
  }
}

class _Notice extends StatelessWidget {
  const _Notice({required this.text, required this.isError});

  final String text;
  final bool isError;

  @override
  Widget build(BuildContext context) {
    final color = isError ? const Color(0xFFAA3038) : Colors.green;
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(10),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.08),
        borderRadius: BorderRadius.circular(8),
        border: Border.all(color: color.withValues(alpha: 0.25)),
      ),
      child: Row(
        children: <Widget>[
          Icon(
            isError ? Icons.error_outline : Icons.info_outline,
            color: color,
          ),
          const SizedBox(width: 8),
          Expanded(
            child: Text(text, style: TextStyle(color: color)),
          ),
        ],
      ),
    );
  }
}
```

android_apps/hsk_mission_lab/lib/coach_api.dart

```
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter_appauth/flutter_appauth.dart';

class SessionUser {
  const SessionUser({required this.displayName, required this.username});

  final String displayName;
  final String username;
}

class LoginSession {
  const LoginSession({required this.token, required this.user});

  final String token;
  final SessionUser user;
}

class ChatMessage {
  const ChatMessage({
    required this.role,
    required this.content,
    this.recordingPath,
    this.source = 'text',
  });

  final String role;
  final String content;
  final String? recordingPath;
  final String source;

  Map<String, String> toJson() => <String, String>{
    'role': role,
    'content': content,
    'source': recordingPath == null ? source : 'voice',
  };
}

class ConversationHistory {
  const ConversationHistory({
    required this.messages,
    this.level,
    this.scenario,
    this.hasMore = false,
    this.nextCursor,
  });

  final List<ChatMessage> messages;
  final int? level;
  final String? scenario;
  final bool hasMore;
  final ConversationCursor? nextCursor;
}

class ConversationCursor {
  const ConversationCursor({
    required this.conversationId,
    required this.id,
    required this.createdAt,
  });

  final String conversationId;
  final String id;
  final String createdAt;
}

abstract class CoachGateway {
  String get baseUrl;
  set baseUrl(String value);

  Future<LoginSession> login(String username, String password);

  Future<LoginSession> loginWithOidc({
```

```
    required String issuer,
    required String clientId,
    required String redirectUrl,
  });

  Future<String> transcribe({
    required String token,
    required List<int> wavBytes,
    required int level,
    required String scenario,
  });

  Future<String> chat({
    required String token,
    required int level,
    required String scenario,
    required List<ChatMessage> messages,
  });

  Future<ConversationHistory> loadLatestConversation({
    required String token,
    ConversationCursor? before,
  });

  void resetConversation();
}

class CoachApi implements CoachGateway {
  CoachApi({
    this.baseUrl = '***',
    this.clientApp = 'unknown',
  });

  @override
  String baseUrl;
  final String clientApp;
  final FlutterAppAuth _appAuth = const FlutterAppAuth();
  String? _conversationId;

  @override
  Future<LoginSession> login(String username, String password) async {
    final payload = await _post('/api/login', <String, Object?>{
      'username': username,
      'password': password,
      'appId': clientApp,
    });
    final user = payload['user'] as Map<String, dynamic>?;
    final token = payload['token'] as String?;
    if (user == null || token == null || token.isEmpty) {
      throw const CoachApiException('           ');
    }
    _conversationId = null;
    return LoginSession(
      token: token,
      user: SessionUser(
        displayName: user['displayName'] as String? ?? username,
        username: user['username'] as String? ?? username,
      ),
    );
  }

  @override
  Future<LoginSession> loginWithOidc({
    required String issuer,
    required String clientId,
    required String redirectUrl,
  }) async {
    final result = await _appAuth.authorizeAndExchangeCode(
      AuthorizationTokenRequest(clientId, redirectUrl,
          issuer: issuer, scopes: const <String>['openid', 'profile']),
    );
    final accessToken = result.accessToken;
    if (accessToken == null || accessToken.isEmpty) {
      throw const CoachApiException('            ');
    }
    final payload = await _post('/api/auth/oidc', <String, Object?>{
      'accessToken': accessToken,
      'appId': clientApp,
```

```
    });
    final user = payload['user'] as Map<String, dynamic>?;
    final token = payload['token'] as String?;
    if (user == null || token == null || token.isEmpty) {
      throw const CoachApiException('             ');
    }
    _conversationId = null;
    return LoginSession(token: token, user: SessionUser(
      displayName: user['displayName'] as String? ?? '  ',
      username: user['username'] as String? ?? '',
    ));
  }

  @override
  Future<String> transcribe({
    required String token,
    required List<int> wavBytes,
    required int level,
    required String scenario,
  }) async {
    final payload = await _post(
      '/api/transcribe',
      <String, Object?>{
        'audio': 'data:audio/wav;base64,${base64Encode(wavBytes)}',
        'locale': 'zh-CN',
        'appId': clientApp,
        'conversationId': _conversationId,
        'level': level,
        'scenario': scenario,
      },
      token: token,
      timeout: const Duration(seconds: 65),
    );
    final transcript = payload['transcript'] as String?;
    if (transcript == null || transcript.trim().isEmpty) {
      throw const CoachApiException('               ');
    }
    _conversationId = payload['conversationId'] as String? ?? _conversationId;
    return transcript.trim();
  }

  @override
  Future<String> chat({
    required String token,
    required int level,
    required String scenario,
    required List<ChatMessage> messages,
  }) async {
    final payload = await _post(
      '/api/chat',
      <String, Object?>{
        'level': level,
        'scenario': scenario,
        'appId': clientApp,
        'conversationId': _conversationId,
        'messages': messages
            .skip(messages.length > 24 ? messages.length - 24 : 0)
            .map((message) => message.toJson())
            .toList(),
      },
      token: token,
      timeout: const Duration(seconds: 45),
    );
    final reply = payload['reply'] as String?;
    if (reply == null || reply.trim().isEmpty) {
      throw const CoachApiException('AI          ');
    }
    _conversationId = payload['conversationId'] as String? ?? _conversationId;
    return reply.trim();
  }

  @override
  Future<ConversationHistory> loadLatestConversation({
    required String token,
    ConversationCursor? before,
  }) async {
    final query = <String, String>{'appId': clientApp, 'limit': '80'};
    if (before != null) {
```

```
    query.addAll(<String, String>{
      'cursorConversationId': before.conversationId,
      'cursorId': before.id,
      'cursorCreatedAt': before.createdAt,
    });
  }
  final payload = await _get(
    '/api/history/latest?${Uri(queryParameters: query).query}',
    token: token,
  );
  _conversationId = payload['conversationId'] as String?;
  final rawMessages = payload['messages'] as List<dynamic>? ?? const [];
  final rawCursor = payload['nextCursor'] as Map<String, dynamic>?;
  final cursor = rawCursor == null
      ? null
      : ConversationCursor(
          conversationId: rawCursor['conversationId'] as String? ?? '',
          id: rawCursor['id']?.toString() ?? '',
          createdAt: rawCursor['createdAt'] as String? ?? '',
        );
  return ConversationHistory(
    level: payload['level'] as int?,
    scenario: payload['scenario'] as String?,
    hasMore: payload['hasMore'] == true && cursor != null,
    nextCursor: cursor,
    messages: rawMessages
        .whereType<Map<String, dynamic>>()
        .map(
          (message) => ChatMessage(
            role: message['role'] as String? ?? 'assistant',
            content: message['content'] as String? ?? '',
            source: message['source'] as String? ?? 'text',
          ),
        )
        .where((message) => message.content.isNotEmpty)
        .toList(),
  );
}

@override
void resetConversation() {
  _conversationId = null;
}

Future<Map<String, dynamic>> _get(
  String path, {
  required String token,
}) async {
  final client = HttpClient()..connectionTimeout = const Duration(seconds: 8);
  try {
    final root = baseUrl.trim().replaceAll(RegExp(r'/+$'), '');
    final request = await client.getUrl(Uri.parse('$root$path'));
    request.headers.set(HttpHeaders.authorizationHeader, 'Bearer $token');
    final response = await request.close().timeout(
      const Duration(seconds: 15),
    );
    final payload =
        jsonDecode(await utf8.decoder.bind(response).join())
            as Map<String, dynamic>;
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw CoachApiException(payload['error'] as String? ?? '         ');
    }
    return payload;
  } on SocketException {
    throw const CoachApiException('         ');
  } on TimeoutException {
    throw const CoachApiException('         ');
  } finally {
    client.close(force: true);
  }
}

Future<Map<String, dynamic>> _post(
  String path,
  Map<String, Object?> body, {
  String? token,
  Duration timeout = const Duration(seconds: 15),
}) async {
```

```
    final client = HttpClient()..connectionTimeout = const Duration(seconds: 8);
    try {
      final root = baseUrl.trim().replaceAll(RegExp(r'/+$'), '');
      final request = await client.postUrl(Uri.parse('$root$path'));
      request.headers.contentType = ContentType.json;
      if (token != null) {
        request.headers.set(HttpHeaders.authorizationHeader, 'Bearer $token');
      }
      request.write(jsonEncode(body));
      final response = await request.close().timeout(timeout);
      final responseBody = await utf8.decoder.bind(response).join();
      final payload = jsonDecode(responseBody) as Map<String, dynamic>;
      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw CoachApiException(
          payload['error'] as String? ?? '       ${response.statusCode}  ',
        );
      }
      return payload;
    } on SocketException {
      throw const CoachApiException(
        '         USB         adb reverse tcp:8040 tcp:8040 ',
      );
    } on TimeoutException {
      throw const CoachApiException('             ');
    } on FormatException {
      throw const CoachApiException('         ');
    } finally {
      client.close(force: true);
    }
  }
}

class CoachApiException implements Exception {
  const CoachApiException(this.message);
  final String message;
  @override
  String toString() => message;
}
```

database/migrations/003_practice_platform.sql

```
CREATE TABLE IF NOT EXISTS hsk_practice_questions (
  id text PRIMARY KEY,
  level smallint NOT NULL CHECK (level BETWEEN 1 AND 6),
  section text NOT NULL CHECK (section IN ('listening', 'reading', 'speaking')),
  question text NOT NULL CHECK (length(question) BETWEEN 1 AND 4000),
  options jsonb NOT NULL CHECK (jsonb_typeof(options) = 'array' AND jsonb_array_length(options) BETWEEN 2 AND 6),
  answer smallint NOT NULL CHECK (answer > 0),
  explanation text NOT NULL DEFAULT '',
  audio_text text NOT NULL DEFAULT '',
  source text NOT NULL DEFAULT 'HSK         ',
  created_by text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  CHECK (answer <= jsonb_array_length(options))
);

CREATE INDEX IF NOT EXISTS idx_hsk_practice_questions_level_section_created
  ON hsk_practice_questions (level, section, created_at DESC);

CREATE TABLE IF NOT EXISTS hsk_practice_attempts (
  id bigserial PRIMARY KEY,
  user_id text NOT NULL,
  question_id text NOT NULL,
  level smallint NOT NULL CHECK (level BETWEEN 1 AND 6),
  section text NOT NULL CHECK (section IN ('listening', 'reading', 'speaking')),
  is_correct boolean NOT NULL,
  attempted_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_hsk_practice_attempts_user_time
  ON hsk_practice_attempts (user_id, attempted_at DESC);
```